



الجامعة السورية الخاصة
SYRIAN PRIVATE UNIVERSITY

University Name: Syrian Private University
Specialization: Software Engineering

VitaMate: A smart lifestyle and health tracking application.

**A Junior Project submitted in partial fulfillment of the
requirements of the Fifth Year, Faculty of Software Engineering,
Syrian Private University.**

Prepared by:

Salam Mohammed Al-Ayash

Amenah Ayman Zaitoun

Supervised by:

Eng. Raghad Al-Hossny

Academic Year: 2025 – 2026

Damascus, Syria

Title Page.....	1
Table of Contents.....	2
Chapter 1 Introduction.....	8
1. Introduction.....	9
2. Overview of the Field and Project Significance.....	9
3. Problem Definition.....	9
4. Project Objectives.....	10
5. Project Scope.....	10
6. Project Significance.....	11
Chapter 2 Basic concepts and literature review.....	12
1. Introduction.....	13
2. Fundamental Concepts.....	15
3. Reference Study.....	16
1. MyFitnessPal.....	16
2. Google Fit.....	16
3. WaterMinder.....	17
4. Sleep Cycle.....	17
5. Habitica.....	18
Chapter 3 Project planning.....	20
1. Introduction.....	21
2. Project charter.....	21
3. The SOW document.....	22
4. Project plan - Gantt chart.....	24

5. Risk Management.....	25
6. Summary.....	26
Chapter 4 System analysis.....	28
1. Software Requirements Specification (SRS).....	29
1.1 Introduction.....	29
1.2 Overall Description.....	29
1.2.1. Product perspective.....	29
1.2.2. Product Functions.....	29
1.2.3. User Classes and Characteristics.....	30
1.2.4. Use Cases.....	30
1.2.4.1 User Use Cases.....	30
1.2.4.2 Use Cases details.....	31
1.2.4.3 Use case diagram.....	35
1.3 Functional requirements.....	36
1.4 Non-functional requirements.....	38
1.5 Constraints Section.....	40
1.5.1 Introduction.....	40
2. Analysis diagram.....	40
2.1 introduction.....	40
2.2 Analysis class diagram.....	41
4. Initial Test Cases.....	41
4.1 Introduction.....	41
4.2 Test cases.....	42
5. RTM.....	44

Chapter 5 System Design.....	48
1. Chapter introduction.....	49
2. Sprint Planning & Backlog.....	49
2.4 Sprint 3: Activity Tracking and Sensor-Based Features.....	50
2.5 Sprint 4: Personal Goals and Insights.....	50
2.6 Sprint 5: Finalization, Testing, and Delivery.....	50
2.7 Backlog Prioritization and Refinement.....	50
2.9 VitaMate Product Backlog.....	51
2.9.1 Sprint 0: Requirements & Planning.....	51
2.9.2 Sprint 1: Core Setup & User Management.....	52
2.9.3 Sprint 2: Daily & Health Tracking.....	52
2.9.4 Sprint 3: Activity Tracking & Sensor-Based Features.....	52
2.9.5 Sprint 4: Goals & Motivation.....	53
2.9.6 Sprint 5: Testing, Refinement & Delivery.....	53
3. Database Design.....	53
3.1 Introduction.....	54
3.2 The ERD (Entity –Relationship Diagram).....	54
3.2.1 ERD Introduction.....	54
3.2.2 ERD diagram schema.....	55
4. Component Diagram.....	56
4.1 Introduction.....	56
4.2 Component diagram.....	56
5. Architecture Design.....	57
5.1 Introduction.....	57
5.2 Architecture diagram.....	57
5.3 Justification for Using the Component-Based Architecture.....	58

5.4 Detailed Behavioral Diagrams.....	58
5.4.1 Water Intake Tracking Diagrams.....	58
5.4.1.1 Sequence diagram.....	59
5.4.2 Nutrition and Calorie Management Diagrams.....	59
5.4.2.1 Sequence diagram.....	60
5.4.2.2 Activity diagram.....	61
5.4.3 Physical Activity Tracking Diagrams.....	61
5.4.3.1 Sequence diagram.....	62
5.4.4 Sleep Tracking Diagrams.....	63
5.4.4.1 Sequence Diagram.....	63
5.4.4.2 Activity diagram.....	64
5.4.5 Personal Goal Management Diagrams.....	64
5.4.5.1 Sequence Diagram.....	65
5.4.5.2 Activity Diagram.....	66
Chapter 6 Implementation.....	67
6.1. Chapter Introduction.....	68
6.1.1. Scope Freeze (Release Scope).....	68
6.2 Technology Stack and Libraries.....	70
6.2.1 Backend Technologies (Django / REST API).....	70
6.2.2 Frontend Technologies (Flutter Mobile Application).....	70
6.2.3 Development and Testing Tools.....	70
6.2.4 Summary Table (Stack Overview).....	71
6.2.5 Table: Tools and Technologies Used in VitaMate.....	71
6.2.6 Tools and Technologies Justification.....	72
6.2.5 Versioning & Dependency Management.....	72

6.3 Backend Implementation.....	73
6.3.1 Project Structure (Backend Tree).....	73
6.3.2 Key Entry Points and Configuration.....	74
6.3.3 Backend Layering (How Responsibilities Are Separated).....	75
6.3.4 Modules Overview.....	75
6.3.5 API Routing Summary (Where Endpoints Are Defined).....	76
6.3.6 Traceability Summary (Features → Code Locations).....	77
6.3.3 Component Summary Table (Backend).....	78
6.3.4 Layering (How Responsibilities Are Separated).....	80
6.4 Frontend Structure (Flutter).....	84
6.6 Testing & Verification.....	89
6.6.1 Purpose and Scope of Testing.....	89
6.6.2 Test Strategy and Levels.....	90
6.6.3 Test Environment and Reproducibility.....	90
6.6.4 Backend Verification (API Tests).....	93
6.6.4.1 Coverage.....	93
6.6.4.2 Test Suite Structure (As Implemented).....	93
6.6.4.3 Execution Results.....	94
6.6.5 Frontend Verification (Flutter Tests).....	94
6.6.5.1 Coverage.....	94
6.6.5.2 Rationale for Mocks.....	94
6.6.5.3 Execution Results.....	95
6.6.6 Performance & Concurrency Verification (Locust).....	95
6.6.6.1 Scenario Definition (“Normal Conditions”).....	95
6.6.6.2 Metrics Collected.....	95
6.6.6.3 Results (from captured evidence).....	96

6.6.7 Traceability Matrix (FR/NFR → Tests → Evidence).....	97
6.6.7.1 FR Traceability (Evidence -backed).....	97
6.6.7.2 NFR Traceability (Evidence -backed).....	97
6.6.8 UI Evidence (Screenshots of the Application).....	97
6.6.9 Known Gaps, Limitations, and Future Improvements.....	106
6.6.10 Summary of Verification Outcomes.....	106
Conclusion & Future Work.....	107

Chapter 1

Introduction

1. Introduction:

In this chapter, we provide a brief overview of our project, highlighting the main problem it aims to address. Additionally, we outline the key objectives and goals that this system seeks to achieve.

2. Overview of the Field and Project Significance:

In today's fast-paced world and with the growing prevalence of unhealthy lifestyles, many individuals struggle to modify their habits and overcome behaviors that negatively impact their physical and mental well-being. This has created an increasing demand for an intelligent health assistant capable of tracking daily routines and supporting users in improving their overall lifestyle.

Based on this need, the VitaMate project was conceived. VitaMate is an intelligent health assistant application designed to monitor and enhance users' well-being holistically by tracking nutrition, water intake, physical activity, sleep quality, medication adherence, and by assisting users in breaking unhealthy habits.

3. Problem Definition:

A large number of individuals struggle to maintain consistent healthy daily habits essential for their physical and mental well-being. These include eating a balanced diet, drinking sufficient amounts of water, engaging in regular physical activity, and maintaining proper sleep schedules.

Neglecting these habits often leads to a gradual decline in overall health and increases the risk of developing chronic conditions such as obesity, hypertension, diabetes, and sleep disorders. It can also negatively affect one's psychological state and daily productivity. Therefore, there is a growing need for an intelligent and comprehensive system that assists individuals in managing their health more effectively by tracking their daily behaviors, providing personalized reminders, and offering analytical insights to help them monitor progress and continuously improve their lifestyle.

The **VitaMate** application addresses this issue by offering a digital platform designed to help users establish sustainable healthy habits and enhance their self-awareness toward better health management.

4. Project Objectives:

General Objective:

The VitaMate project aims to develop an intelligent mobile application that helps users adopt a healthier lifestyle by monitoring nutrition, physical activity, sleep patterns, water intake, and eliminating unhealthy habits.

Specific Objectives:

1. Track daily caloric intake and food quality based on the user's health condition and personal goals.
2. Monitor water consumption levels and encourage users to maintain proper daily hydration.
3. Track users' physical activity and monitor their exercise routines regularly.
4. Assist users in overcoming unhealthy habits such as smoking or excessive sugar consumption.
5. Regulate sleep schedules and improve sleep quality to achieve a balanced and healthy rest pattern.
6. Provide weekly and monthly health reports that include analytics and personalized recommendations for lifestyle improvement.

5. Project Scope:

In-Scope:

1. Development of a comprehensive mobile application designed to monitor and promote a healthy lifestyle.
2. A secure login system ensuring user data protection and privacy.
3. A PostgreSQL database for storing and managing user information and health-related habits.
4. A notification system to remind users of their daily goals and scheduled activities.
5. User-friendly and interactive interfaces that provide clear health metrics and progress indicators, supported by a motivational point-based system.

6. Project Significance:

Practical Benefits:

1. Assisting users in adopting a balanced and healthy lifestyle through consistent behavior tracking.
2. Regularly monitoring nutrition, physical activity, and sleep to enhance overall well-being.
3. Promoting health awareness and encouraging users to make positive changes in their daily habits.
4. Supporting users in maintaining commitment to healthy routines and achieving their personal goals.

Scientific Benefits:

1. Implementing modern technologies such as Flutter and Django in the development of intelligent health applications.
2. Gaining practical experience in designing and building an integrated health monitoring system.
3. Utilizing user-generated data to analyze health patterns and enabling future integration of artificial intelligence to improve recommendation accuracy.

Chapter 2

Basic concepts and literature review

1. Introduction:

This chapter provides a comprehensive overview of the fundamental concepts underlying the **Vitamate** project — an intelligent system designed to monitor and improve users' lifestyles.

It covers the theoretical foundations related to healthy habit tracking, health data management, self-motivation, and behavioral analysis.

Understanding these concepts is essential to demonstrate how the system supports users in enhancing their health and making informed decisions that improve their overall well-being.

2. Fundamental Concepts:

■ Healthy Lifestyle Tracking

Healthy lifestyle tracking refers to the process of monitoring daily behaviors that influence an individual's health, such as nutrition, physical activity, sleep, and water intake.

This tracking helps identify positive and negative habits and analyze their impact on overall well-being.

Lifestyle tracking systems are effective tools for increasing self-awareness and supporting users in setting measurable goals and monitoring their progress.

■ Health Balance and Prevention

The concept of balanced health emphasizes maintaining harmony between various factors such as proper nutrition, regular exercise, and sufficient rest.

This balance plays a key role in preventing chronic diseases such as obesity, diabetes, and cardiovascular conditions.

The **Vitamate** project aims to support this balance by monitoring daily health indicators and alerting users when adjustments in behavior are needed.

■ Self-Motivation and Behavioral Change

Self-motivation is a crucial factor in maintaining healthy habits over time.

Studies have shown that continuous feedback and symbolic rewards can significantly enhance adherence and encourage long-term improvement.

Vitamate applies a **points-based reward system** to motivate users to achieve their daily goals and foster a sense of accomplishment.

■ Smart Reminders and Daily Follow-Up

Health reminder systems help users organize their time and maintain consistency in their daily habits.

Reminders are effective tools for reducing forgetfulness and improving adherence to healthy routines, such as drinking water regularly, exercising, or maintaining proper sleep patterns.

In **Vitamate**, reminders serve as a core feature to ensure daily engagement and sustained progress toward personal health goals.

■ Health Behavior Analysis

Health behavior analysis involves interpreting recorded user data to assess progress and lifestyle trends.

Through this analysis, the system identifies strengths and weaknesses in user habits and suggests targeted improvements.

Behavioral analysis tools enable users to visualize the impact of their habits on overall health, encouraging consistent and informed self-improvement.

■ Health Awareness

Health awareness refers to an individual's understanding of how lifestyle choices affect physical and mental well-being.

Promoting health awareness is one of the primary objectives of modern health systems, as it empowers users to make knowledge-based decisions instead of random or reactive ones.

Vitamate enhances health awareness by presenting users with clear, understandable insights about their daily performance and long-term progress.

■ Health Data Management

Health data management involves collecting, organizing, and analyzing user-related information to create a comprehensive overview of health status.

This process must be accurate, reliable, and privacy-conscious.

Data management forms the foundation of any intelligent health system, enabling accurate reporting and data-driven recommendations for users.

■ Reports and Feedback

Periodic reports allow users to evaluate their progress toward their health goals. Feedback plays a key role in improving performance and modifying behavior by providing clear indicators of both progress and areas needing attention.

Vitamate offers weekly and monthly reports summarizing users' health performance, encouraging continuous lifestyle improvement.

■ Privacy and Health Data Security

Privacy is a critical concern in digital health systems due to the sensitivity of personal user information.

Such systems must guarantee secure data handling and prevent unauthorized access.

Vitamate emphasizes the importance of data protection by securely storing user information and maintaining confidentiality according to modern technical standards.

■ The Future of Smart Health Systems

Modern digital health solutions are increasingly integrating intelligent analysis and personalized recommendations.

With advancements in artificial intelligence, future systems can predict user behavior and provide more precise, adaptive health guidance.

The **Vitamate** project represents an early step toward this direction by collecting and analyzing data to support evidence-based health decisions.

3. Reference Study

Selected Reference Applications:

A set of the most widely used and recognized global health and lifestyle applications were selected based on their popularity, functionality, and effectiveness in promoting healthy habits. The selected applications include:

1. MyFitnessPal
2. Google Fit
3. WaterMinder
4. Sleep Cycle
5. Habitica

Analysis of Similar Applications:

MyFitnessPal:

Main Purpose:

Tracking users' calorie intake and dietary habits.

Core Features:

- a. Logging daily meals and food items.
- b. Calculating calories per meal in relation to the daily goal.
- c. Providing detailed reports and graphical insights about nutritional progress.

Key Advantages:

- a. A user-friendly interface with access to a vast food database.
- b. Integration with fitness tracking devices for activity synchronization.



Google Fit:

Main Purpose:

Comprehensive monitoring of users' physical activity and overall health through motion, step count, calorie tracking, and sleep analysis using mobile sensors or wearable devices.

Core Features:

- a. Step counting.
- b. Calorie estimation.
- c. Distance measurement.



- d. Daily activity level tracking.
- e. Sleep and heart rate monitoring (when connected to smart devices).
- f. Providing daily and weekly health indicators such as *Heart Points*.
- g. Data synchronization with other applications (e.g., MyFitnessPal, Sleep as Android).

Key Advantages:

- a. Strong integration with **Android Wear** and other smart devices.
- b. Well-structured dashboards that visualize health progress.
- c. Ability to customize daily health goals based on user preferences.

WaterMinder:

Main Purpose:

Encouraging users to maintain adequate hydration throughout the day.

Core Features:

- a. Determining daily water intake goals based on user weight and activity level.
- b. Sending periodic reminders to drink water.
- c. Displaying hydration levels in graphical form.

Key Advantages:

- a. Smart reminders based on real-time user activity.
- b. Customizable notifications according to user preferences.



Sleep Cycle:

Main Purpose:

Monitoring and analyzing users' sleep quality and patterns.

Core Features:

- a. Recording sleep duration and restfulness.
- b. Using sensors to analyze body movements during sleep.
- c. Offering recommendations to improve sleep quality and determine optimal wake-up times.

Key Advantages:

- a. Detailed reports on sleep quality and ideal wake-up moments.
- b. Simple interface allowing users to monitor sleep trends over time.



Habitica:

Main Purpose:

Motivating users to maintain healthy and productive habits through a gamified experience.

Core Features:

- a. Creating daily habit lists and health-related tasks.
- b. Transforming task completion into XP points and virtual rewards.
- c. Allowing users to share progress and achievements with friends or teams for added motivation.



Key Advantages:

- a. Incorporating **Gamification** principles to make habit tracking engaging and enjoyable.
- b. Visually appealing interface that encourages consistent participation.

Summary:

Feature / Application	Habitica	Sleep Cycle	WaterMinder	Google Fit	MyFitnessPal	VitaMate
Nutrition Tracking	X	X	X	X	✓	✓
Water Consumption Tracking	X	X	✓	X	X	✓
Physical Activity Tracking	X	X	X	✓	✓	✓
Sleep Monitoring	X	✓	X	X	X	✓
Progress Reports	X	X	X	X	X	✓
Motivational Reward System (Points)	✓	✓	✓	✓	X	✓
Lifestyle Score Calculation	✓	X	X	X	X	✓
Interactive and User-Friendly Interface	X	X	X	X	X	✓
Habits System	✓	✓	✓	✓	✓	✓
Synchronization With External Systems	X	X	X	✓	✓	X
Custom Goal Creation	✓	✓	X	X	✓	✓

Chapter 3

Project planning

1. Introduction:

Chapter three focuses on the comprehensive planning process required to manage and execute the project effectively. It includes the **Project Charter**, which defines the vision, objectives, and project scope, followed by the **Statement of Work (SOW)** that outlines the detailed tasks and deliverables. The chapter also introduces the **Gantt Chart**, representing the project timeline and task scheduling.

Additionally, the chapter discusses the **development methodology**, the **tools and technologies used**, and the **risk management plan** responsible for identifying potential challenges and proposing strategies to mitigate them. This structured planning framework ensures a clear roadmap for successful and efficient project implementation

2. Project charter:

A project charter is a formal document that serves as an official authorization for the start of a project. It acts as a reference point throughout the project, providing a clear understanding of the project's purpose and establishing a foundation for decision-making and project governance.

<i>Item</i>	<i>Details</i>
Project Title	A Health Assistant System That Helps Users Follow a Healthy Lifestyle
Project Start Date	20-10-2025
Project End Date	20-12-2025
Project Supervisor	Eng. Raghad Al-Hosani
Project Objectives	To develop an intelligent health assistant application that supports users in adopting and maintaining a healthy lifestyle by tracking nutrition, water intake, physical activity, and sleep habits. The system will include a motivational reward-based points mechanism to encourage consistency and long-term engagement.
Methodology / Approach	1. Define the project scope and main objectives. 2. Gather and analyze user requirements. 3. Create and prioritize the Product Backlog. 4. Conduct Sprint Planning to select tasks for each development cycle.

Roles and responsibilities:	5. Design the system architecture and database structure.		
	6. Develop core functionalities through iterative Sprints using Scrum.		
	7. Perform continuous testing to ensure quality and fix issues.		
	8. Conduct Sprint Review to demonstrate progress and receive feedback.		
	9. Run Sprint Retrospective to evaluate performance and improve processes.		
	10. Document all development phases and prepare final delivery outputs.		

3. The SOW document:

Statement of Work is a comprehensive document that defines the scope of work for the Vitamate project. It outlines tasks, deliverables, responsibilities, and timeline. The document provides a clear understanding of the project objectives and success criteria.

Project Title	Vitamate – Smart Health & Wellness Assistant
Project Description & Objectives	Vitamate is a smart health tracking application that helps users improve their health habits by tracking water intake, sleep hours, calorie consumption, and physical activity. It provides reminders, reports, and insights through a friendly mobile interface. The system integrates with fitness sensors (Google Fit / Apple Health) to provide accurate real-time data and personalized recommendations.
Project Scope	The project includes developing a mobile application that enables users to monitor health habits such as water intake, sleep, calories, and workouts. It provides notifications, weekly/monthly

	reports, dashboards, and sensor integration. The system includes backend API, mobile frontend, user authentication, and deployment.
Project Goals	• Improve user lifestyle through habit tracking and data insights. • Provide accurate real-time statistics using wearable sensors. • Offer personalized recommendations and reminders. • Deliver weekly/monthly dashboards and progress analytics.
Project Deliverables	• Vitamate mobile application (Android/iOS). • Admin dashboard. • Reports & analytics dashboards. • User authentication & profile management. • Physical activity tracking & sensor integration. • Documentation package & final presentation.
Technology & Tools	• Frontend: Flutter • Backend: Django REST Framework • Database: PostgreSQL • APIs: Google Fit / Apple Health • Authentication: JWT • Version Control: Git / GitHub
Assumptions	• Continuous availability of the development team. • Availability of required development environment. • Access to Google Fit & Apple Health APIs. • Regular meetings with project manager for supervision and feedback.
Project Resources	<i>Human Resources:</i> <i>Raghad al Hosny- project management</i> <i>Salam Al Ayash- SE Developer</i> <i>Amenah Zaitoun- SE Developer</i>
Schedules	• Project Start Date: 20.10.2025 • Sprint 0: 27.10.2025 • Sprint 1: 15.11.2025 • Sprint 2: 22.11.2025 • Sprint 3: 29.11.2025 • Sprint 4: 6.12.2025 • Sprint 5: 13.12.2025 • Project Finish Date: 20.12.2025
Milestones & Seminars	• First Seminar: 18.11.2025 • Second Seminar: TBA • Final Presentation: Upon project delivery

5. Risk Management:

the process of identifying, evaluating, and mitigating potential risks that could impact the success of a project or a team.

Field	RK-01	RK-02	RK-03	RK-04
Risk Title	Small team size (2 members)	Scope creep / unclear requirements	New technologies (sensors + analytics)	Backend–Frontend separation (integration risk)
Risk Description	If one member is delayed/unavailable, overall progress is highly affected.	Unclear/changing requirements may cause rework and wrong implementation.	Limited experience with sensor integration and analytics may delay delivery.	Separation may cause mismatched APIs, delays, and integration failures.
Raised Date	20/10/2025	28/10/2025	13/11/2025	15/11/2025
Likelihood	4	4	3	4
Impact	5	4	4	4
Score (L×I)	20	16	12	16
Owner	Both team members	PM/Supervisor + Team	Technical lead (assigned dev)	Both devs (BE+FE)
Tracking Frequency	Weekly (each sprint review)	Weekly + after change request	Weekly + early spike checkpoints	2× per week + sprint demo
Early Warning / Trigger	Missed task > 2 days / 1 member unavailable > 48h	New features after Sprint 0 / backlog grows > 20%	Prototype not working by Sprint 2 end / blocked > 3 days	FE blocked waiting endpoints > 24h / repeated API changes

Mitigation Plan	Cross-training + shared docs; pair programming on critical modules; prioritize MVP.	Requirements freeze end of Sprint 0; change-request log; MoSCoW; weekly validation.	Early spike prototype; split learning; proven plugins; document integration steps.	Agree API contract (OpenAPI/Swagger); mock server; sprint integration checkpoints; early integration tests.
Contingency Plan	Freeze non-critical features; reduce scope to MVP; re-plan sprint backlog.	Defer advanced features; deliver only must-have for release.	Use manual input fallback; keep analytics as basic stats only.	Stub endpoints temporarily; freeze API last 3 days; ship minimal stable endpoints.
Status	Monitoring	Mitigating	Mitigating	Active
Next Review	15/11/2025	15/11/2025	22/11/2025	22/11/2025

6. Summary

- Effective project management is essential for the successful development of the Vitamate system.
- It ensures organized work, clear structure, and timely completion within scope and deadlines.
- By applying Agile Scrum, the team can identify risks early and improve the quality of the final product.
- Overall, strong project management supports delivering a high-quality system that meets user needs.

Chapter 4

System analysis

1. Software Requirements Specification (SRS):

1.1 Introduction

This document specifies the Software Requirements Specification (SRS) for the VitaMate system.

The purpose of this document is to provide a precise and complete description of the system's functional and non-functional requirements. These requirements define the expected behavior, constraints, and quality attributes of the system to ensure correct implementation, verification, and future scalability.

This SRS serves as a contractual reference between stakeholders, developers, and evaluators, ensuring a consistent and unambiguous understanding of system requirements.

1.2. Overall Description

1.2.1. *Product Perspective*

VitaMate is a standalone mobile health and lifestyle tracking system supported by a backend server and a centralized database.

The system is designed using a client–server architectural model, where responsibilities are clearly separated to improve maintainability, scalability, and security.

The major architectural components are defined as follows:

The mobile application, implemented using Flutter, acts as the client interface through which users interact with the system.

The backend server, implemented using Django REST Framework, is responsible for executing business logic and providing RESTful APIs.

A PostgreSQL database is used to store persistent system and user data.

External health service APIs (Pedometer) are used to obtain sensor-based activity data when available.

1.2.2. *Product Functions*

At a high level, the VitaMate system shall provide the following functions:

User account registration and authentication.

Tracking of daily nutrition and caloric intake.

Monitoring of daily water consumption.

Tracking of physical activities and workouts.

Monitoring and analysis of sleep duration and quality.

Management of unhealthy habits.

Delivery of automated notifications and reminders.

Generation of periodic health reports.

Provision of a motivational, points-based reward mechanism.
Administrative monitoring and system management.

1.2.3. User Classes and Characteristics

User Class	Description
Regular User	An end user who uses the system to track health-related activities, receive reminders, and view reports.
Administrator	A privileged user responsible for monitoring system usage, managing data, and ensuring system stability.

1.2.4. Use Cases

1.2.4.1. User Use Cases:

Use Case Name	Description
Create Account	Allows the user to create a new personal account.
Manage User Account	Allows the user to manage account settings and personal information.
Enter Personal Health Data	Allows the user to enter health-related data (height, weight, age, gender, activity level).
Set Personalized Goals	Allows the user to define customized health and lifestyle goals.
Track Water Intake	Allows the user to track daily water consumption.
Track Sleep	Allows the user to track sleep duration and sleep patterns.
Track Nutrition & Calories	Allows the user to log meals and monitor daily caloric intake.
Track Physical Activity	Allows the user to manually track physical activities.
Track General Health Level	Allows the user to monitor the overall health level based on aggregated lifestyle data (nutrition, activity, sleep, water, and habits).

Manage Notifications & Reminders	Allows the user to manage reminders and notifications.
View Reports & Statistics	Allows the user to view health reports, analytics, and progress indicators.

1.2.4.2. Use Cases details:

UC-03 — Enter Personal Health Data:

Field	Details
<i>Primary Actor</i>	User
<i>Goal</i>	Enter basic health data (height, weight, age, gender, activity level) for personalization.
<i>Preconditions</i>	Account exists + user is logged in.
<i>Trigger</i>	Opening onboarding/health questionnaire screen.
<i>Main Success Scenario</i>	1) System displays the health data form. 2) User enters required values. 3) System validates ranges and formats. 4) App sends data to backend. 5) Backend stores data and returns success. 6) User is redirected to the home/dashboard.
<i>Alternative Flows</i>	A1: Invalid values → System requests correction and shows validation error.
<i>Exception Flows</i>	E1: Network/server failure → Error message and retry.
<i>Postconditions</i>	Health profile data is stored successfully.

UC-05 — Track Water Intake:

Field	Details
<i>Primary Actor</i>	User
<i>Goal</i>	Log water intake and update user's progress/indicators.
<i>Preconditions</i>	User is logged in.

<i>Trigger</i>	User opens “Water” and selects “Add Intake”.
<i>Main Success Scenario</i>	1) User opens the Water tracking screen. 2) User enters a water amount (or selects a preset if available). 3) System validates the amount (must be positive). 4) App sends the water log to the backend. 5) Backend stores the record and returns success. 6) App displays the updated daily progress.
<i>Alternative Flows</i>	— (None)
<i>Exception Flows</i>	E1: Invalid input (zero/negative/empty) → Show validation message. E2: Server/network failure → Show error and allow retry.
<i>Postconditions</i>	Water record is saved and progress displayed to the user is updated.

UC-06 — Track Sleep:

<i>Field</i>	Details
<i>Primary Actor</i>	User
<i>Goal</i>	Log sleep session and compute duration; update progress/points if applicable.
<i>Preconditions</i>	User is logged in.
<i>Trigger</i>	User opens Sleep and selects “Add Sleep”.
<i>Main Success Scenario</i>	1) User opens Sleep screen. 2) User enters sleep start and wake-up time. 3) System validates time range (including midnight crossing). 4) App sends record to backend. 5) Backend stores record and computes duration. 6) App displays duration and updated progress/points.
<i>Alternative Flows</i>	A1: Invalid time range → Show error and request correction. A2: User cancels → No data saved.
<i>Exception Flows</i>	E1: Server/network error → Show error and retry.
<i>Postconditions</i>	Sleep record stored and indicators updated.

UC-07 — Track Nutrition & Calories (Manual Nutrition Data + Food DB):

Field	Details
Primary Actor	User
Goal	Add meals and daily calorie totals; allow manual nutrition entry and store it in the food database.
Preconditions	User is logged in.
Trigger	User opens Nutrition and selects “Add Meal”.
Main Success Scenario	1) User opens Nutrition module. 2) User adds a meal and enters meal details (name/time/etc.). 3) User enters nutrition data manually (calories/macros as designed). 4) System saves the meal record. 5) System saves the entered nutrition item into the Food Database for future use. 6) System updates and displays daily totals.
Alternative Flows	A1: Manual nutrition entry → Save meal + persist nutrition data into Food DB (same as above).
Exception Flows	E1: Missing/invalid nutrition fields → Request correction. E2: Server/network error → Error and retry.
Postconditions	Meal stored; daily totals updated; Food DB updated with manual entry.

UC-08 — Track Physical Activity (Steps):

Field	Details
Primary Actor	User
Goal	Track activity via step counting (walking/exercises) and record/display progress.
Preconditions	User is logged in + Physical Activity permission is requested/granted.
Trigger	User opens Activity module.

Main Success Scenario	1) User opens Activity module. 2) App requests Physical Activity permission (if not granted). 3) User selects “Walking” or “Exercises”. 4) System tracks step count. 5) App displays steps and progress. 6) (Optional) Send step log to backend for storage/reporting.
Alternative Flows	A1: Permission denied → Show guidance; tracking disabled (or allow manual entry if supported).
Exception Flows	E1: Sensor reading failure → Show error and allow retry. E2: Server/network failure (if sending data) → Error handling per system policy.
Postconditions	Steps tracked/displayed; logs stored if enabled.

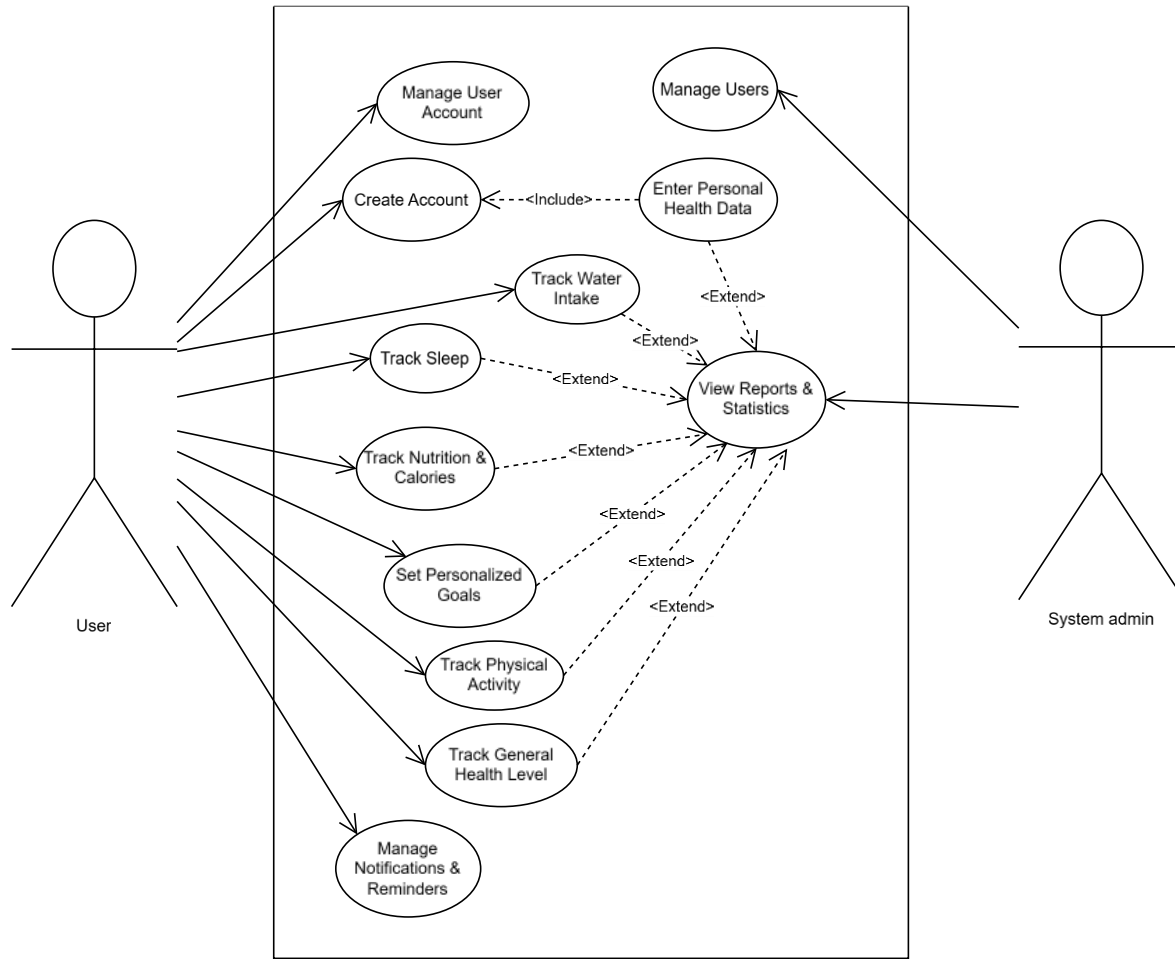
UC-09 — View Overall Health Level:

Field	Details
Primary Actor	User
Goal	View an overall health level/summary based on stored tracking data.
Preconditions	User is logged in.
Trigger	User opens Dashboard/Health Level.
Main Success Scenario	1) User opens Dashboard. 2) App requests overall summary from backend. 3) Backend returns computed indicators. 4) App displays the overall health level and summary.
Alternative Flows	— (None)
Exception Flows	E1: Summary cannot be generated (insufficient data/service error) → Show failure state. E2: Server/network error → Error and retry.
Postconditions	Summary displayed (if available).

UC-10 — Manage Notifications & Reminders:

<i>Field</i>	<i>Details</i>
<i>Primary Actor</i>	User
<i>Goal</i>	Enable/disable and schedule reminders on the device.
<i>Preconditions</i>	User is logged in + notifications permission available on device.
<i>Trigger</i>	User opens Reminders/Notifications settings.
<i>Main Success Scenario</i>	1) User opens reminders settings. 2) User enables/disables reminders. 3) User sets time/frequency. 4) System saves preferences. 5) System schedules local notifications on device.
<i>Alternative Flows</i>	A1: Permission denied → Prompt user to enable notifications in OS settings.
<i>Exception Flows</i>	E1: Scheduling failure → Show error and allow retry.
<i>Postconditions</i>	Reminder preferences saved; notifications scheduled.

1.2.4.3. Use case diagram:



1.3. Functional requirements

FR ID	Requirement Name	Academic Specification
FR-01	User Account Creation	The system shall allow users to create a personal account.
FR-02	Personal Health Data Entry	The system shall require users to enter personal health data (height, weight, age, gender, physical activity level) through a questionnaire as a prerequisite for completing account creation.

FR-03	Lifestyle Score Calculation	The system shall calculate a lifestyle score based on the entered questionnaire data and inform the user of their health needs.
FR-04	Physical Activity Tracking	The system shall provide functionality to track physical activity and support users in improving it.
FR-05	Physical Activity Goal Setting	The system shall allow users to activate and define a physical activity goal.
FR-06	Activity Method Selection	The system shall allow users to select their preferred method for achieving the physical activity goal (e.g., gym, running, home workouts).
FR-07	Step Counter Activation	The system shall allow users to activate a step counter to record daily steps.
FR-08	Daily Step Target Calculation	The system shall calculate the required daily number of steps to achieve the defined physical activity goal.
FR-09	Exercise Logging	The system shall allow users to log exercises performed during the day.
FR-10	Burned Calories Calculation	The system shall calculate burned calories using the MET (Metabolic Equivalent of Task) rate.
FR-11	Daily Calorie Adjustment	The system shall adjust the recommended daily caloric intake based on physical activity level and defined goals.
FR-12	Weekly Activity Hours Definition	The system shall allow users to define a minimum number of weekly physical activity hours required to achieve their goal.
FR-13	Physical Activity Reminders	The system shall allow users to configure the number and timing of physical activity reminders.
FR-14	Dietary Goal Definition	The system shall allow users to define their dietary goal (weight loss, weight gain, maintenance, muscle building).
FR-15	Daily Calorie Requirement Calculation	The system shall calculate the daily caloric intake required to achieve the defined dietary goal.

FR-16	Meal Logging	The system shall allow users to log meals and assign caloric values to each meal.
FR-17	Food Database Support	The system shall provide a food database to assist in calculating meal calories.
FR-18	Daily Consumed Calories Calculation	The system shall calculate the total daily consumed calories based on logged meals.
FR-19	Calorie Intake Monitoring	The system shall monitor user compliance with assigned daily calories and notify the user in case of excess or deficiency.
FR-20	Daily Water Requirement Calculation	The system shall calculate the required daily water intake based on user weight and gender.
FR-21	Water Intake Adjustment	The system shall adjust the daily water requirement based on physical activity performed on the same day.
FR-22	Water Tracking Activation	When water tracking is activated, the system shall display periodic reminders to drink water.
FR-23	Water Intake Logging	The system shall allow users to log consumed water and track progress toward the daily goal.
FR-24	Water Intake Motivation	The system shall award motivational points to encourage users to meet their daily water intake goal.
FR-25	Sleep Improvement Activation	The system shall allow users to activate sleep improvement functionality.
FR-26	Sleep Data Entry	The system shall allow users to define their typical sleep duration and identify sleep patterns (deep, moderate, light, interrupted).
FR-27	Sleep Duration Calculation	The system shall calculate required sleep duration and recommend appropriate sleep and wake-up times.
FR-28	Sleep Schedule Management	The system shall allow users to define and update sleep and wake-up times.
FR-29	Reward Points System	The system shall award points for each health component and calculate an overall health reward score.

FR-30	Penalty Points Deduction	The system shall deduct points when the user violates defined health goals.
FR-31	Health Reports Generation	The system shall generate periodic comprehensive reports reflecting the user's overall health compliance.
FR-32	Goal Progress Tracking	The system shall track user compliance and progress toward achieving defined personal goals.

1.4. Non-functional requirements

NFR ID	Category	Requirement (Refined)	Acceptance Criteria (Pass/Fail)	Verification Method	Evidence to Attach (from current code)
<i>NFR-01</i>	Performance	The system shall respond within ≤ 2 seconds under normal operating conditions.	95% of key requests $\leq 2s$; max $\leq 3s$.	Test	Postman timings (screenshots/table).
<i>NFR-02</i>	Performance	Support concurrent access without major response-time degradation.	With 20 concurrent users: avg latency increase $\leq 30\%$, error rate $< 2\%$.	Test/Analyses	Load test report (Locust/JMeter) or documented "Target" rationale.
<i>NFR-04</i>	Security	Passwords shall be stored using secure hashing.	No plain-text passwords; user creation uses create_user + password validators enabled.	Inspection	serializers.py (create_user) + settings.py (password validators).
<i>NFR-05</i>	Privacy	Prevent unauthorized access and isolate each user's personal/health data.	JWT-protected APIs + IsAuthenticated + querysets filtered by request.user + sensitive fields are read-only.	Inspection/ Test	settings.py/urls.py (JWT routes) + views.py (IsAuthenticated + request.user filtering) + serializers.py (read-only fields).
<i>NFR-06</i>	Reliability	Availability target during demo/testing period.	$\geq 99\%$ availability during a defined test window (e.g., one week) or defined demo hours.	Analysis	Defined measurement window + simple log/assumptions.
<i>NFR-07</i>	Reliability	Recover gracefully without loss of critical user data.	No loss of previously saved logs; consistent writes.	Analysis/Inspection	Write-consistency note + optional failure scenario test.

NFR-08	Usability	Clear, consistent UI suitable for non-technical users.	5-user quick test: ≥80% complete 3 tasks unaided.	Demo/Test	Task script + results summary + UI screenshots.
NFR-09	Usability	Minimize interaction steps for common tasks.	Water ≤3 taps; Sleep ≤4 steps; Meal ≤5 steps.	Inspection/Demo	“Task → steps” table + UI screenshots.
NFR-10	Scalability	Support growth without major architectural changes.	Stateless REST + pagination + documented DB indexes on user/date.	Analysis/Inspection	Architecture note + pagination example + index migrations snippet.
NFR-11	Maintainability	Modular architecture for maintenance and enhancements.	Clear module separation in Flutter + Django apps.	Inspection	Project tree + module boundaries note.
NFR-12	Compatibility	Operate on Android and iOS.	Built and run on Android + iOS (device/simulator) with versions documented.	Demo	Run screenshots + OS versions.
NFR-13	Interoperability	Integrate with Pedometer/Physical Activity .	Steps are read after permission is granted; graceful handling if denied.	Demo/Test	steps_controller.dart + AndroidManifest.xml + steps evidence.
NFR-14	Data Integrity	Ensure accuracy and consistency of stored health data.	Input validation + consistent units + consistent storage.	Inspection/Test	Validation rules + negative test examples + schema/migrations if available.
NFR-15	Availability	Accessible except scheduled maintenance.	Maintenance window defined + maintenance message during downtime.	Analysis	Maintenance policy paragraph + optional screenshot.

1.5. Constraints Section

1.5.1. Introduction

The system is subject to several constraints that must be considered during development and deployment. These constraints represent limitations related to technology, resources, time, and external dependencies, and they directly influence the design and implementation of the VitaMate system.

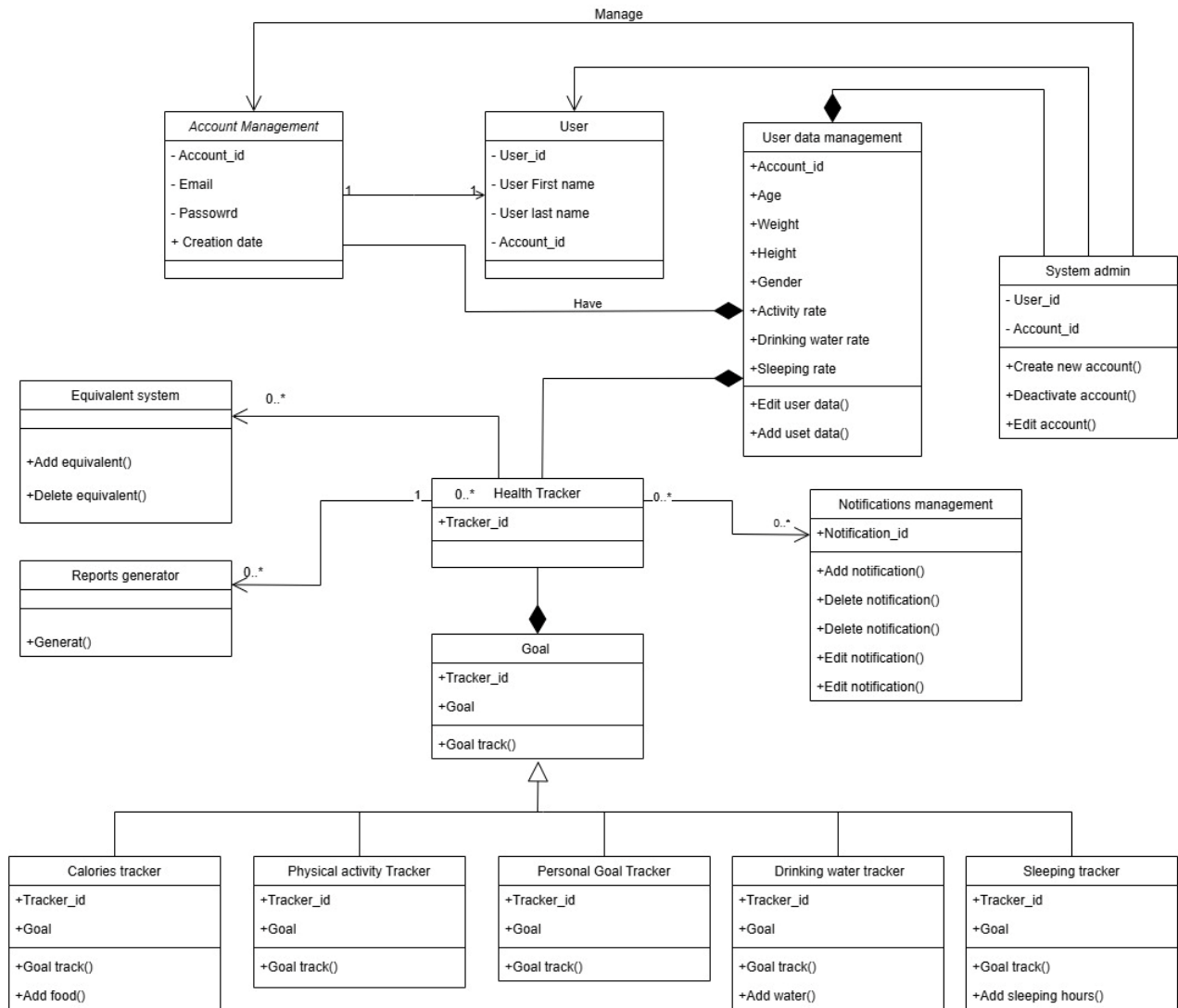
Constraint ID	Type	Description
C-01	Technical	The system must operate on Android and iOS platforms.
C-02	Security	The system must adhere to data privacy standards due to the sensitivity of health information.
C-03	Connectivity	An active internet connection is required for synchronization and report generation.
C-04	Resource	System performance must consider limited device resources such as memory and battery usage.
C-05	Development	The development schedule is constrained by the academic semester timeframe.
C-06	Legal	The system shall comply with regulations related to data protection (e.g., GDPR-like standards if applicable).

2. Analysis diagram

2.1. introduction

The Analysis Diagram represents the conceptual structure of the VitaMate system during the requirements analysis phase. It identifies the main classes, their responsibilities, and their relationships, without considering implementation details. The purpose of this diagram is to model the problem domain and ensure a clear understanding of how system components interact to fulfill the functional requirements. By visualizing associations, aggregations, and dependencies, the diagram provides a high-level abstraction that supports accurate system design and forms a foundation for subsequent architectural and class diagrams.

2.2. Analysis class diagram



4. Initial Test Cases

4.1. Introduction

The Initial Test Cases provide an early verification approach to ensure that the core functional requirements of the VitaMate system are fulfilled. These test cases outline expected inputs, execution steps, and corresponding outputs for essential scenarios such as authentication, data tracking, and goal management. By defining these tests at an early stage, the project ensures traceability between requirements

and validation activities, helping identify potential issues before implementation and guiding quality assurance throughout development.

4.2. Test cases

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions	Linked (UC/NFR)
TC-01	User Registration with Valid Data	User not registered; registration screen accessible	Email: user1@test.com, Password: Str0ngPass!123	1) Open registration. 2) Enter email/password/confirm. 3) Submit.	New account is created + success shown + user is redirected to onboarding (health data entry) or home as per flow.	One new user is created in DB.	NFR-04, UC-03 (entry after signup)
TC-02	Registration with Existing Email	Account already exists with same email	Email: user1@test.com, Password: Str0ngPass!123	1) Open registration. 2) Enter existing email + valid password. 3) Submit.	Registration rejected + "email already in use" message + no new user created.	User count unchanged.	NFR-14
TC-03	Successful Login	User account exists; correct credentials	Email: user1@test.com, Password: Str0ngPass!123	1) Open login. 2) Enter credentials. 3) Login.	Authentication succeeds + redirect to dashboard + access/refresh tokens stored in Secure Storage.	Active session established (tokens stored).	NFR-05
TC-04	Add Meal (Manual Nutrition Entry) + Update Daily Calories + Save to Food DB	User is logged in	Meal: "Lunch", Time: 13:00, Calories: 600, Protein: 25g, Carbs: 70g, Fat: 20g	1) Open Nutrition. 2) Add Meal. 3) Manually enter nutrition values. 4) Save.	Meal saved + daily calories increase by +600 + nutrition item is added/updated in Food Database for future use.	Meal record created + daily total updated + Food DB updated.	UC-07, NFR-14
TC-05	Log Water Intake and Update Progress	User is logged in	Water amount: 250 ml	1) Open Water Tracking. 2) Enter 250 ml. 3) Save.	New water record stored + displayed progress increases	New WaterLog created	UC-05, NFR-14

TC-06					by 250 ml + success feedback.	for the user.	
	Step Tracking with Physical Activity Permission (Success)	User logged in; device supports step counter	Permission: Granted; steps in 60s: ≥ 10	1) Open Steps/Activity. 2) Grant permission when prompted. 3) Start tracking and wait 60 seconds.	Steps value > 0 is displayed + (if persistence exists) daily step log is created/updated.	Steps displayed; daily record updated via upsert if applicable.	NFR-13, UC-08
TC-07	Log Sleep Session (Midnight Crossing)	User is logged in	Start: 23:00, End: 07:00	1) Open Sleep Tracking. 2) Enter start/end times. 3) Save.	Sleep record saved + duration calculated correctly = 8 hours.	Sleep record created with duration = 8h.	UC-06, NFR-14
TC-08	Reward Points Updated on Condition Fulfillment (As Implemented)	Points feature enabled; condition can be met	Before: Points = X	1) Perform actions to meet the condition (as supported). 2) Trigger points evaluation (screen/button/automatic flow as implemented).	Points updated: Points = X + N (N defined by system) + UI reflects the new points.	Points value updated + (if available) PointsTransaction saved.	(Gamification)
TC-09	Generate Weekly Health Report	User has 7 days of data (water/sleep/nutrition/steps)	Range: last 7 days	1) Open Reports. 2) Select weekly report. 3) Request report.	Weekly summary is generated/displayed (at minimum: steps avg/total, water total, sleep avg, calories total) without errors.	Report displayed/retrieved successfully.	(Reporting), NFR-14
TC-10	Unauthorized Request without Token	User not authenticated	—	1) Send request to a protected feature (e.g., add water) without JWT.	Request rejected with 401 Unauthorized.	No record created.	NFR-05

TC-11	Cross-User Data Access Attempt	Two users exist (A and B)	Token of A + request for B's data	1) Login as user A. 2) Attempt to read/modify B's records.	Rejected with 403 Forbidden or 404 Not Found (per design); no data leakage.	No leakage; no unauthorized change.	NFR-05
TC-12	Invalid Input (Water = 0)	User is logged in	Water amount: 0 ml	1) Open Water Tracking. 2) Enter 0. 3) Save.	Validation error shown; record is not saved.	No new WaterLog created.	NFR-14

5. RTM

5.1. Introduction

The Requirements Traceability Matrix (RTM) is used to link system requirements with their corresponding test cases. It ensures that every requirement is verified, prevents missing functionality, and supports quality assurance by providing clear traceability throughout the development process.

Requirement ID	Description	Type	Priority	Test Case ID	Verification Method	Status
FR-01	The system shall allow users to create a new account.	Functional	High	TC-01	Test	Pending
FR-02	The system shall allow users to enter personal health data.	Functional	High	TC-01, TC-03	Inspection / Test	Pending
FR-03	The system shall calculate a lifestyle score.	Functional	High	TC-10	Analysis / Test	Pending
FR-04	The system shall track physical activity.	Functional	High	TC-06	Test	Pending

FR-05	The system shall allow users to set activity goals.	Functional	Medium	TC-08	Test	Pending
FR-06	The system shall allow users to select a preferred activity method.	Functional	Low	TC-06	Test	Pending
FR-07	The system shall support step tracking.	Functional	Medium	TC-06	Test	Pending
FR-08	The system shall calculate required daily steps.	Functional	Medium	TC-06	Analysis / Test	Pending
FR-09	The system shall allow users to log exercises.	Functional	Medium	TC-06	Test	Pending
FR-11	The system shall calculate burned calories.	Functional	High	TC-06	Analysis / Test	Pending
FR-12	The system shall adjust daily caloric intake.	Functional	High	TC-04	Analysis / Test	Pending
FR-13	The system shall send physical activity reminders.	Functional	Medium	TC-09	Demonstration / Test	Pending
FR-14	The system shall classify user weight status.	Functional	High	TC-04	Analysis / Test	Pending
FR-15	The system shall allow users to set dietary goals.	Functional	High	TC-04, TC-08	Test	Pending
FR-16	The system shall calculate daily	Functional	High	TC-04	Analysis	Pending

	calorie requirements.					
FR-17	The system shall allow users to log meals.	Functional	High	TC-04	Test	Pending
FR-18	The system shall provide a food database.	Functional	Medium	TC-04	Inspection / Test	Pending
FR-19	The system shall calculate daily consumed calories.	Functional	High	TC-04	Analysis	Pending
FR-20	The system shall monitor caloric intake.	Functional	High	TC-04	Demonstration / Test	Pending
FR-21	The system shall allow users to update weight.	Functional	Medium	TC-04	Test	Pending
FR-22	The system shall calculate water intake needs.	Functional	High	TC-05	Analysis	Pending
FR-23	The system shall adjust water needs based on activity.	Functional	Medium	TC-05	Analysis	Pending
FR-24	The system shall send water reminders.	Functional	Medium	TC-05	Demonstration / Test	Pending
FR-25	The system shall track water intake.	Functional	High	TC-05	Test	Pending
FR-26	The system shall award motivational water points.	Functional	Low	TC-09	Test	Pending

FR-27	The system shall allow users to activate sleep improvement.	Functional	Medium	TC-07	Test	Pending
FR-28	The system shall allow users to enter sleep data.	Functional	High	TC-07	Test	Pending
FR-29	The system shall calculate sleep duration.	Functional	High	TC-07	Analysis / Test	Pending
FR-30	The system shall allow sleep schedule adjustments.	Functional	Medium	TC-07	Test	Pending
FR-31	The system shall award reward points.	Functional	Medium	TC-09	Test	Pending
FR-32	The system shall deduct penalty points.	Functional	Medium	TC-09	Test	Pending
FR-33	The system shall generate health reports.	Functional	High	TC-10	Test	Pending
FR-34	The system shall configure goal duration and reminders.	Functional	Medium	TC-08, TC-09	Test	Pending
FR-35	The system shall track goal progress.	Functional	High	TC-09	Test	Pending

Chapter 5

System Design

1. Chapter introduction

The development of the VitaMate system was carried out using the Agile Scrum methodology, which enabled an iterative and incremental approach to system design and implementation. Instead of constructing the entire system in a single phase, the development process was divided into multiple sprints, each delivering a functional software increment based on prioritized user stories. This chapter presents the system design and illustrates how the architectural models, database structures, and core modules evolved throughout the sprint cycle. The sprint planning, execution, and review ensured continuous refinement of the design and supported adaptability to changing requirements, resulting in a scalable, maintainable, and user-centered system.

2. Sprint Planning & Backlog

The development schedule of the VitaMate system was structured according to the Agile Scrum methodology, where the project lifecycle was divided into six major sprints. These sprints were planned and scheduled as illustrated in the Gantt chart, allowing for incremental delivery and continuous refinement. The Product Backlog served as the central repository of all user requirements and features, from which sprint tasks were selected based on priority, feasibility, and dependencies.

2.1. Sprint 0 – Requirements Analysis and Planning

The initial sprint focused on establishing the foundational understanding of the system. Key activities included requirements elicitation, problem domain analysis, creation of the SRS document, and initial design modeling. During this sprint, the project scope, goals, and architectural direction were defined, and the overall sprint plan was formulated.

2.2. Sprint 1 – Core System Setup and User Management

This sprint concentrated on building the fundamental structure of the system. Major deliverables included the initial UI design, project architecture setup, database creation, and implementation of user authentication and profile management. Basic tracking functionality and initial API configuration were also introduced.

2.3. Sprint 2 – Daily and Health Tracking Features

Sprint 2 extended the system with essential health monitoring functions, including sleep tracking, water intake logging, calorie tracking, notifications, and activity steps tracking. The dashboard and recommendation features were developed to support daily habit monitoring.

2.4. Sprint 3 – Activity Tracking and Sensor-Based Features

This sprint focused on advanced activity tracking supported by sensor data. Features such as workout tracking, calories burned estimation, and enhanced statistical dashboards were delivered in this phase.

2.5. Sprint 4 – Personal Goals and Insights

Sprint 4 introduced goal-oriented features designed to support user motivation and behavior change. Custom goals, goal reminders, integrated statistics, and personalized dashboards were implemented to enhance user engagement and progress monitoring.

2.6. Sprint 5 – Finalization, Testing, and Delivery

The final sprint aimed at stabilizing and optimizing the system. Comprehensive testing, validation against requirements, bug fixing, documentation, and preparation for project delivery were completed. The sprint ensured that the final product aligned with stakeholder expectations.

2.7. Backlog Prioritization and Refinement

The Product Backlog was continuously reviewed and updated through sprint reviews and feedback loops. Items were prioritized based on user value, technical effort, and dependency mapping. Higher priority features—such as authentication, daily tracking, and activity logging—were scheduled earlier, while advanced features like goal integration and reporting were planned in later sprints.

2.8. Outcome of Sprint Planning

This structured sprint allocation allowed for:

- controlled incremental development
- timely delivery of functional increments
- early risk identification
- frequent adaptation to change
- improved alignment with project objectives

2.9. VitaMate Product Backlog

2.9.1. Sprint 0 – Requirements & Planning

Goal: Define system scope and prepare for development.

ID	User Story	Priority
PB-01	As a stakeholder, I want to define the system scope so that project objectives are clearly established.	High
PB-02	As a developer, I want to collect user requirements so that the SRS document can be created.	High
PB-03	As a system analyst, I want to model use cases so that system behavior is understood.	High
PB-04	As a designer, I want to propose the initial system architecture so that design can begin.	Medium
PB-05	As a team, I want to plan sprints so that the work is organized and scheduled.	High

2.9.2. Sprint 1 – Core System & User Management

Goal: Build base structure and authentication.

ID	User Story	Priority
PB-06	As a user, I want to create an account so that I can access the system.	High
PB-07	As a user, I want to log in securely so that my data remains protected.	High
PB-08	As a user, I want to update my profile so that my personal data is accurate.	Medium
PB-09	As a system, I want a database structure so that persistent storage is supported.	High
PB-10	As a user, I want a home interface so that I can navigate the system easily.	High

PB-11	As a user, I want basic tracking initialization so that I can start logging data.	Medium
--------------	---	--------

2.9.3. Sprint 2 – Daily & Health Tracking

Goal: Core lifestyle tracking features.

ID	User Story	Priority
PB-12	As a user, I want to track my sleep duration so that I can improve my sleep cycle.	High
PB-13	As a user, I want to log my daily water intake so that I can reach hydration targets.	High
PB-14	As a user, I want to track my calories so that I can monitor my nutrition.	High
PB-15	As a user, I want notification reminders so that I do not miss daily targets.	Medium
PB-16	As a user, I want to track steps so that I can monitor my activity.	High
PB-17	As a user, I want initial statistics visualization so that I can understand my progress.	Medium

2.9.4. Sprint 3 – Activity Tracking & Sensor-Based Features

Goal: Physical activity tracking and advanced sensing.

ID	User Story (Updated)	Priority
PB-18	As a user, I want step tracking (pedometer-based) so that I can monitor my daily activity level.	High
PB-19	As a user, I want to log workouts manually (type + duration) so that I can document exercise sessions.	Medium
PB-20	As a user, I want calories burned estimation based on available activity inputs (steps and/or duration) so that I can understand workout intensity.	Medium

PB-21	As a system, I want Physical Activity sensor integration (Activity Recognition/Pedometer) so that step tracking is automated.	High
PB-22	As a user, I want dashboard updates so that I can see activity insights (steps progress, trends).	Medium

2.9.5. Sprint 4 – Goals & Motivation

Goal: Personal goals, reward system, and insights.

ID	User Story	Priority
PB-23	As a user, I want to set personal goals so that I can improve my lifestyle.	High
PB-24	As a user, I want goal reminders so that I stay motivated.	Medium
PB-25	As a user, I want reward points so that I feel encouraged to meet targets.	High
PB-26	As a user, I want goal statistics so that I can measure my achievements.	Medium
PB-27	As a user, I want a personalized dashboard so that insights are tailored to my behavior.	Medium

2.9.6. Sprint 5 – Testing, Refinement & Delivery

Goal: Finalization and deployment.

ID	User Story	Priority
PB-28	As a tester, I want to verify system functionality so that requirements are met.	High
PB-29	As a developer, I want to fix defects so that the system becomes stable.	High
PB-30	As a team, I want to document the system so that delivery is complete.	High
PB-31	As a stakeholder, I want a final demo so that delivery can be evaluated.	High

3. Database Design

3.1. Introduction

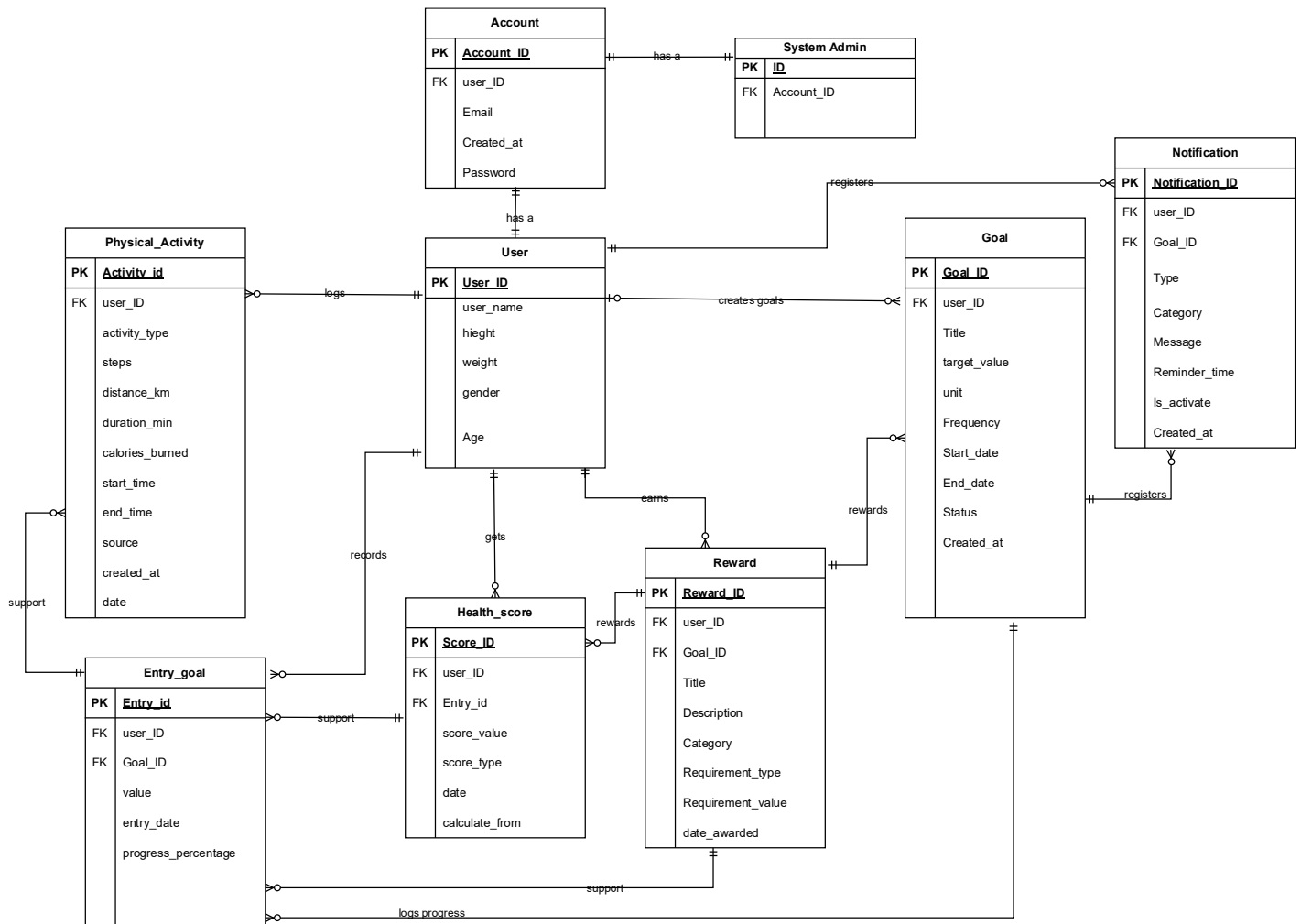
The Database Design section provides the logical representation of the data model used in VitaMate. It explains the Entity–Relationship Diagram (ERD), identifies key entities and their relationships, and presents how user and health-related data are structured, stored, and accessed. The design ensures data integrity, consistency, and efficient retrieval, forming the basis for the relational schema and supporting system requirements for tracking, analysis, and reporting.

3.2. The ERD (Entity–Relationship Diagram)

3.2.1. ERD Introduction

The Entity–Relationship Diagram (ERD) illustrates the logical data model of the VitaMate system. It defines the main entities, their attributes, and the relationships between them, serving as the blueprint for database design. Through this diagram, data dependencies are clarified and redundancy is minimized, ensuring consistency, integrity, and efficient storage of user and health-related information. The ERD supports translation of system requirements into relational tables and enforces constraints that maintain data quality throughout the system lifecycle.

3.2.2.ERD diagram schema



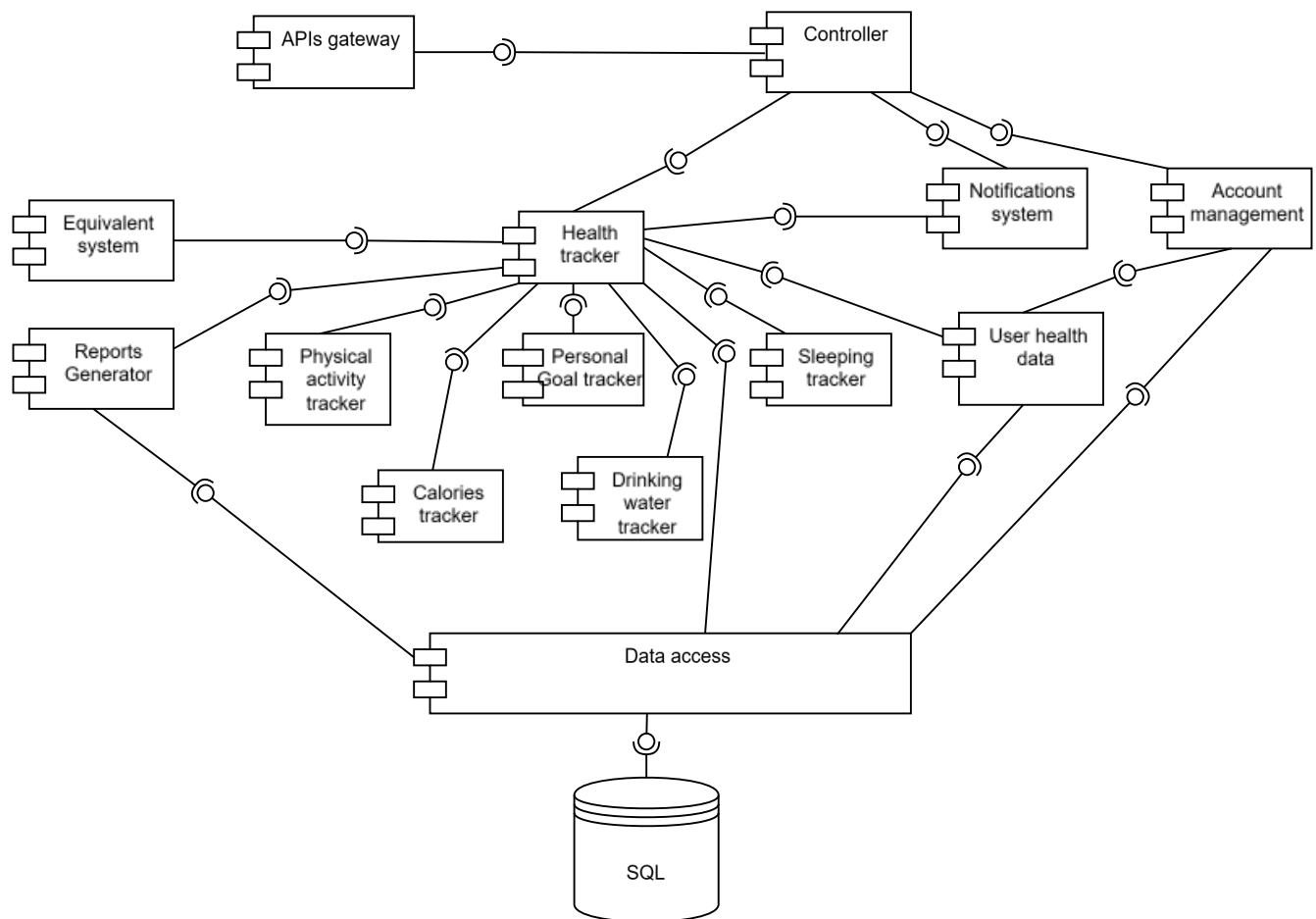
ERD - Schema

4. Component Diagram

4.1. Introduction

The Component Diagram illustrates the modular structure of the VitaMate system by decomposing it into independent and functional components. This section describes how each component contributes to system behavior, clarifies communication interfaces, and highlights the separation of concerns between modules such as authentication, tracking, reporting, and goal management. The diagram supports clear organization of system responsibilities and facilitates parallel development and maintenance.

4.2. Component diagram

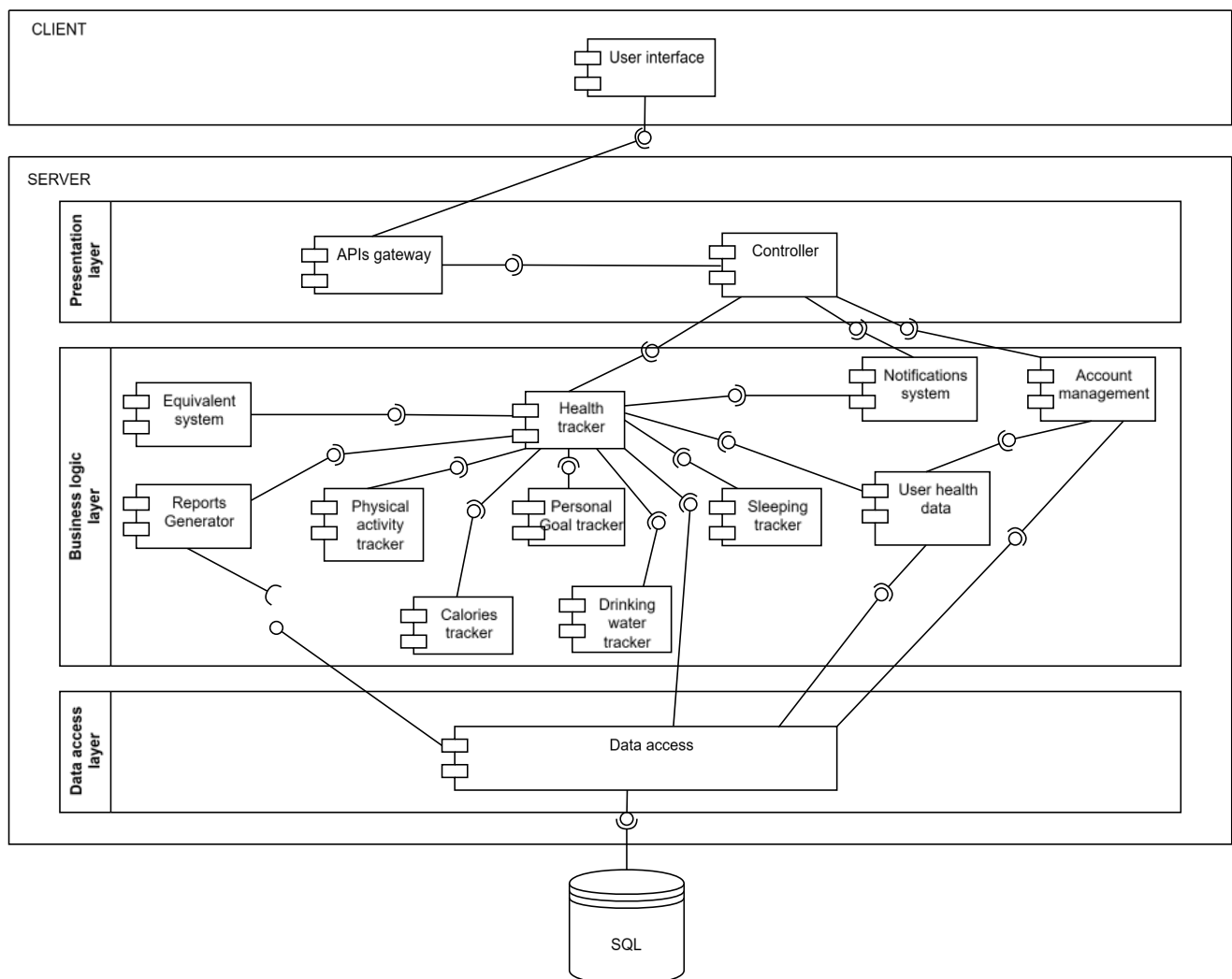


5. Architecture Design

5.1. Introduction

The Architecture Design section presents the high-level structure of the VitaMate system, illustrating how major components interact to achieve the required functionality. It defines the client-server architecture, identifies the system layers, and explains the flow of data between the mobile application, backend services, and database. This architectural model ensures scalability, maintainability, and efficient integration with external APIs, providing a solid foundation for subsequent detailed design and implementation.

5.2. Architecture diagram



5.3. Justification for Using the Component-Based Architecture

The component-based architecture was selected for the VitaMate system due to its ability to support modularity, scalability, and maintainability. By decomposing the system into independent components—such as authentication, tracking modules, goals and rewards, reporting, and external integration—the architecture enables clear separation of responsibilities and reduces interdependencies.

This structure allows individual components to be developed, tested, and refined in parallel, which aligns effectively with the iterative nature of Agile Scrum. Furthermore, updates or enhancements can be introduced with minimal impact on other modules, ensuring adaptability to evolving requirements.

The architecture also improves reliability and integration by enabling controlled communication through well-defined interfaces between the mobile application, backend services, and database. This not only enhances reusability and fault isolation but also supports future expansion, such as adding new tracking features or integrating external

5.4. Detailed Behavioral Diagrams

This section presents the detailed behavioral diagrams for the core functional modules of the VitaMate system, namely water intake tracking, physical activity tracking, sleep tracking, nutrition and calorie management, and personal goal management.

Each module is documented using a sequence diagram and a corresponding activity diagram to describe the interaction between the user interface, backend services, database, and the point-based evaluation system.

These diagrams refine the high-level use cases defined in the SRS and clarify the internal processing logic required to fulfill the system's functional requirements.

5.4.1. Water Intake Tracking Diagrams

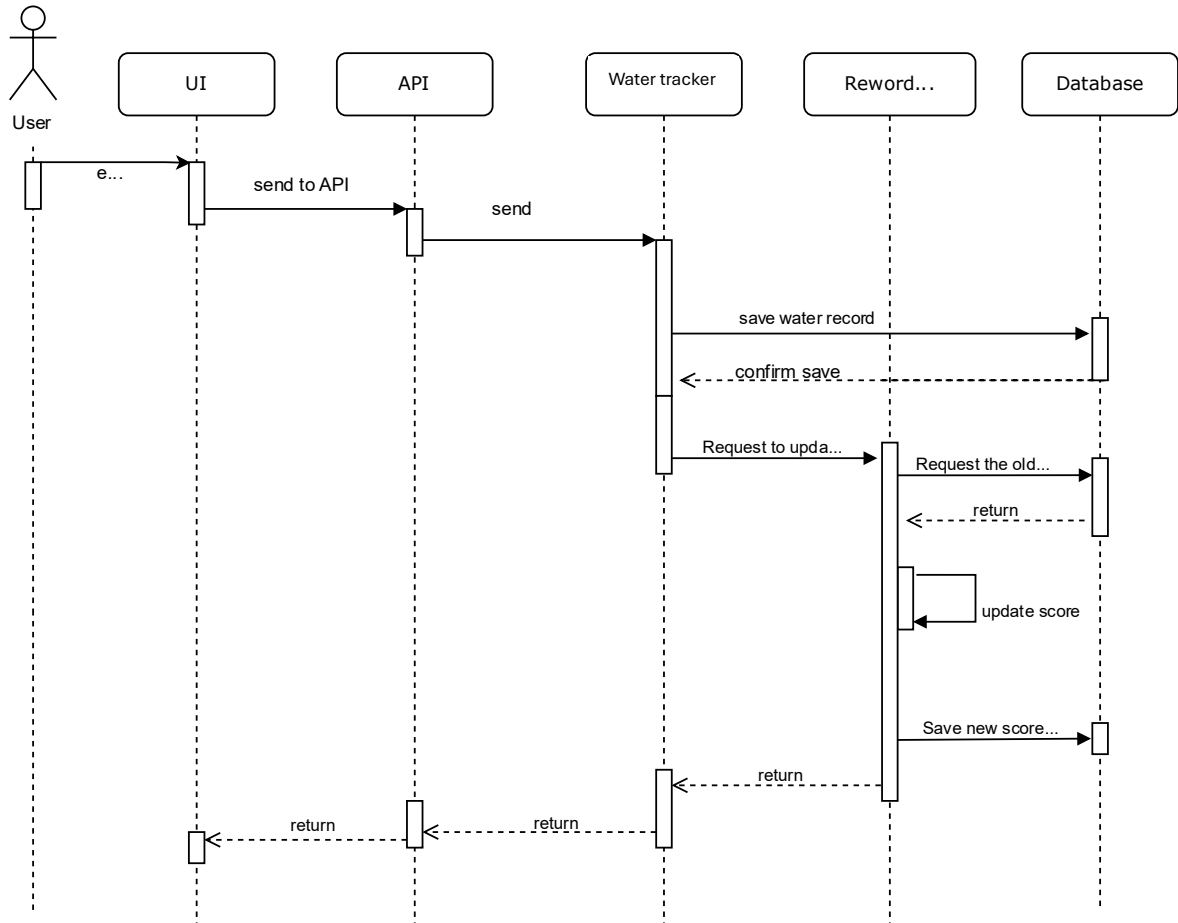
The **Water Intake Tracking Sequence Diagram** illustrates the interaction flow when the user logs a water intake event.

It shows how the mobile UI sends the entered amount to the backend API, how the Water Tracker component validates and stores the record, and how the Point System updates the user's hydration-related score based on progress toward the daily water target.

The **Water Intake Tracking Activity Diagram** models the control flow for this process from the user's perspective.

It describes the main steps of entering the water amount, validating the input, sending the data to the system, storing the record in the database, recalculating the user's daily progress and score, and finally updating the user interface with the new hydration status.

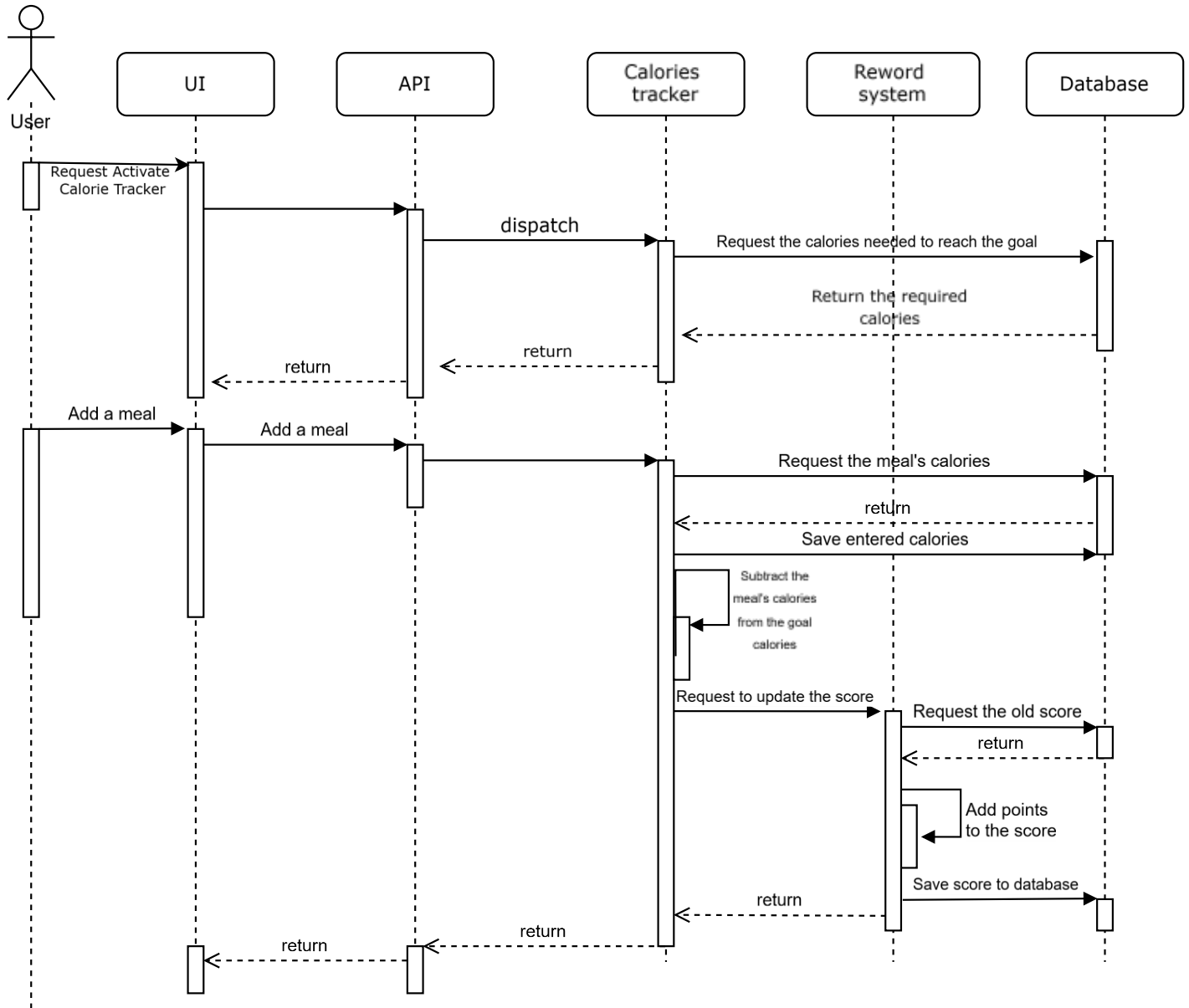
5.4.1.1. Sequence diagram:



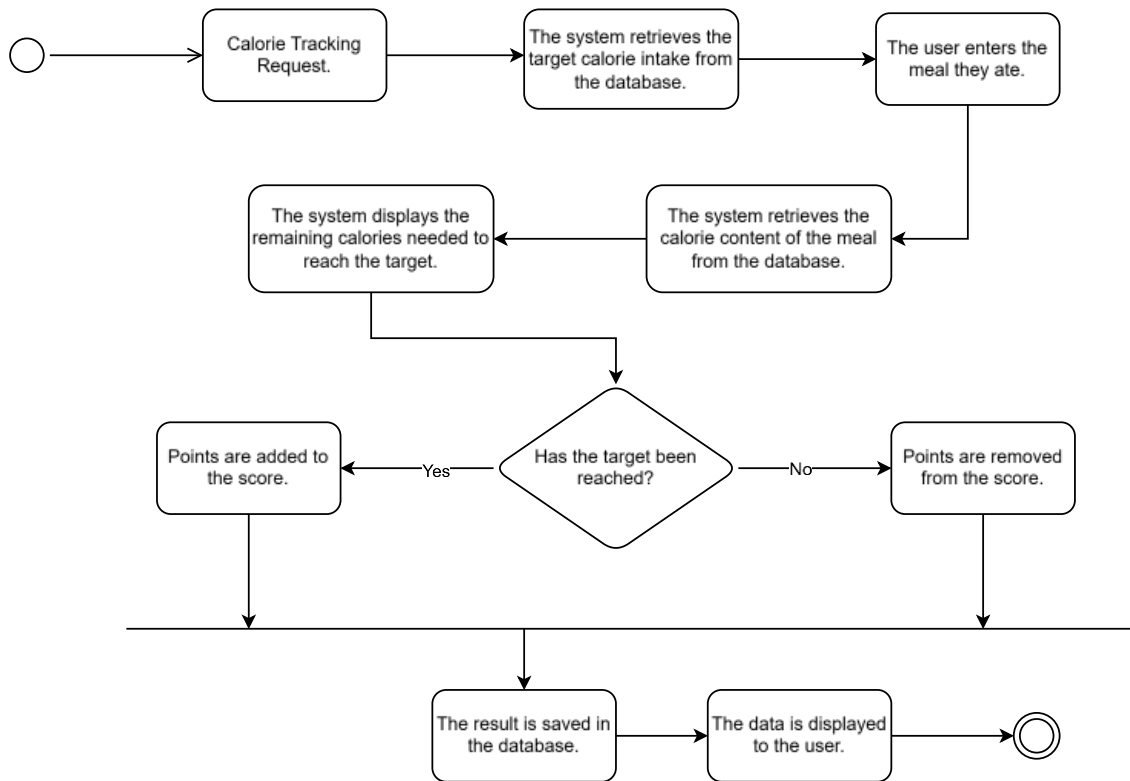
5.4.2. Nutrition and Calorie Management Diagrams

The Nutrition Sprint included user stories for meal logging and calorie computation. The diagrams clarify backend interactions with food databases and point-system logic. They were iteratively refined during backlog grooming as dietary goals became better defined.

5.4.2.1. Sequence diagram:



5.4.2.2. Activity diagram:

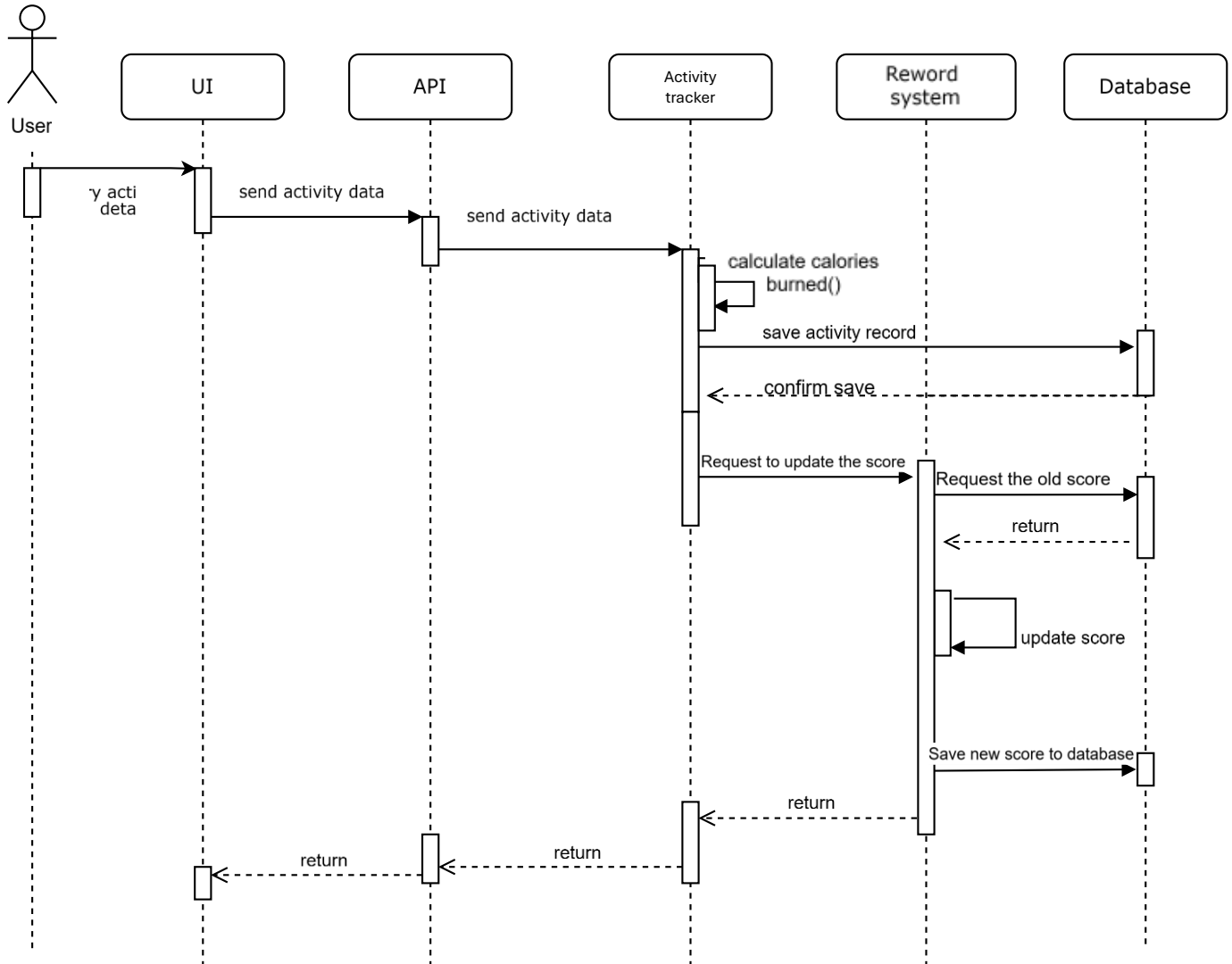


5.4.3. Physical Activity Tracking Diagrams

The **Physical Activity Tracking Sequence Diagram** details the message flow for registering a physical activity session such as walking or running.

The diagram captures how the UI forwards activity data (type, duration, distance) to the API, how the Activity Tracker computes burned calories (using MET-based formulas when applicable), persists the session, and triggers the Point System to adjust the lifestyle score according to the user's activity goals.

5.4.3.1. Sequence diagram:



5.4.4. Sleep Tracking Diagrams

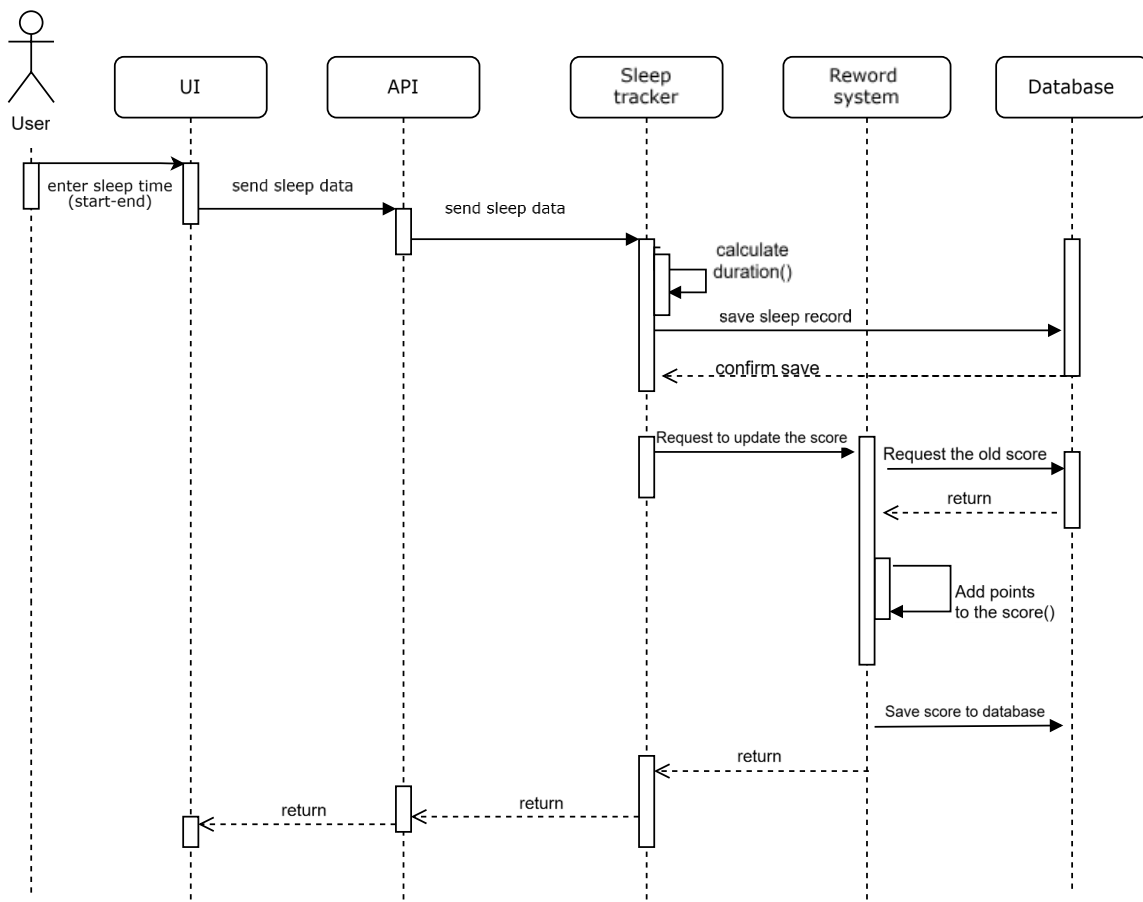
The **Sleep Tracking Sequence Diagram** describes how the system processes a logged sleep session.

Starting from the user entering sleep and wake-up times, the diagram shows how the Sleep Tracker calculates the sleep duration, stores the session in the database, and invokes the Point System to compare the actual duration with the predefined sleep target and update the user's score accordingly.

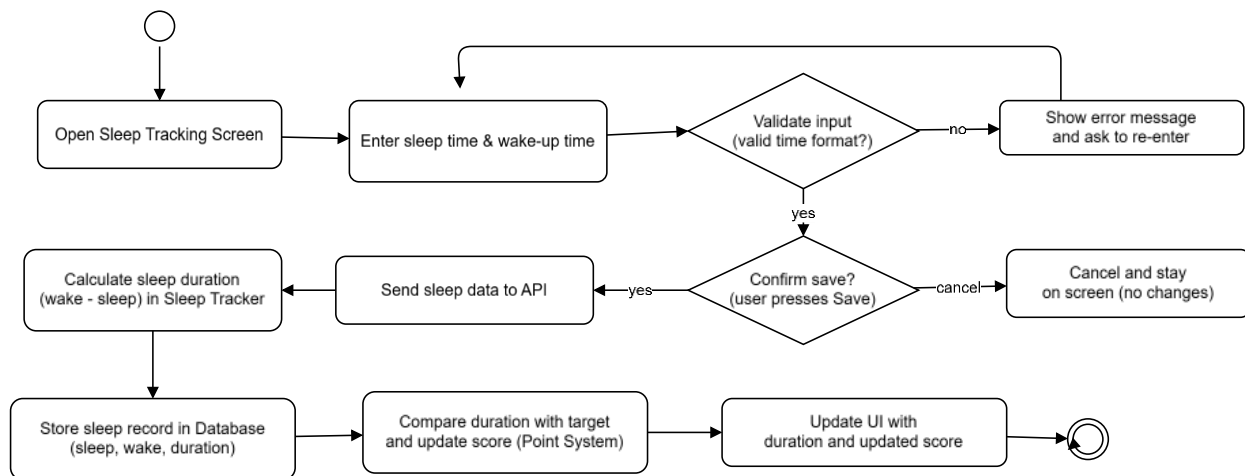
The **Sleep Tracking Activity Diagram** provides a detailed view of the control flow for this scenario.

It covers opening the sleep tracking screen, entering time values, validating the input, handling user cancellation or invalid entries, sending valid data to the backend, computing the duration, saving the sleep record, performing the target comparison, and updating the UI with the calculated duration and new score.

5.4.4.1. Sequence Diagram:



5.4.4.2. Activity diagram:



5.4.5. Personal Goal Management Diagrams

The **Personal Goal Management Sequence Diagram** focuses on the lifecycle of custom goals defined by the user, such as a water target, sleep duration, or activity time.

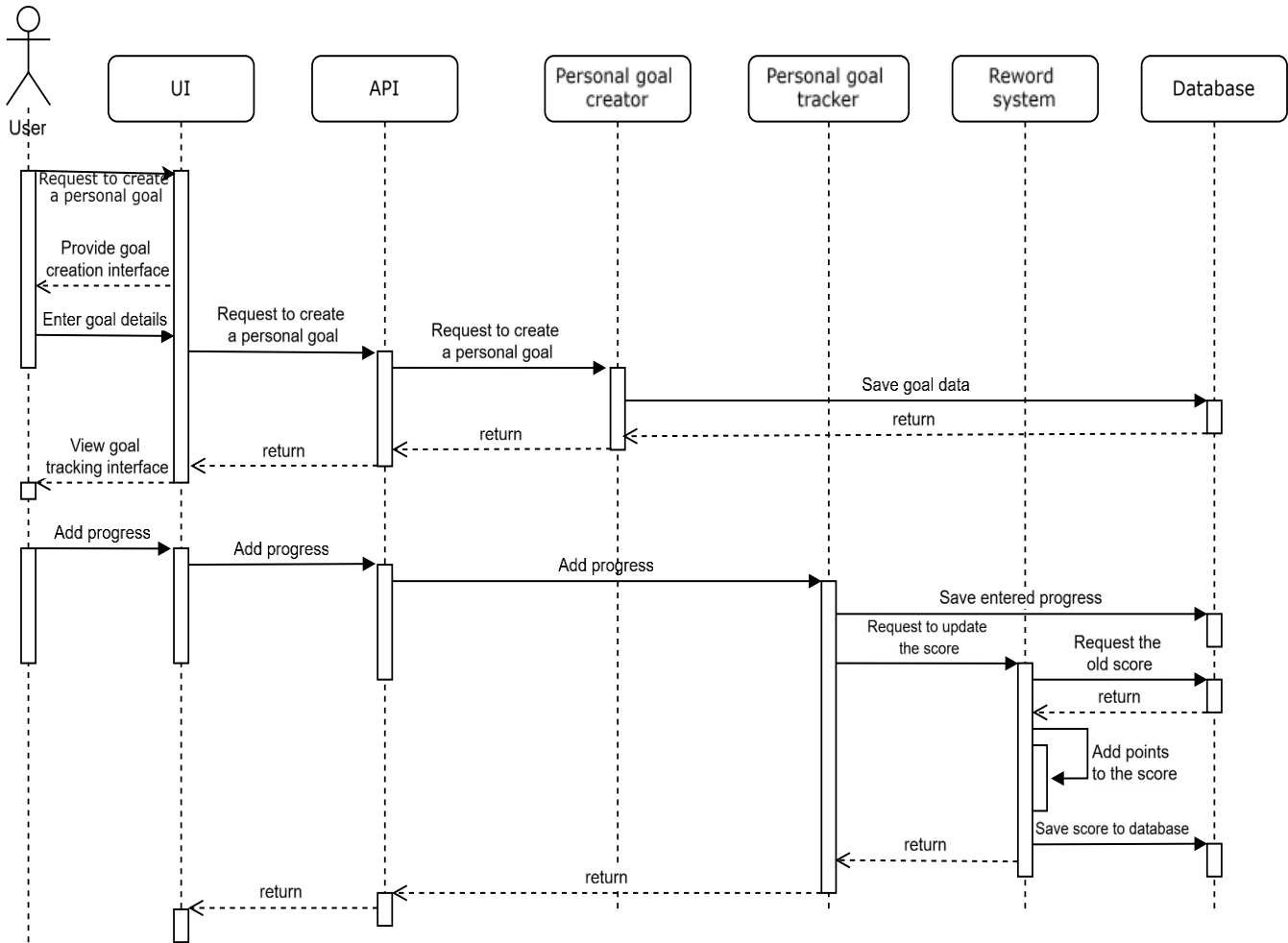
The diagram shows how the UI submits a new goal to the backend, how the Goal Management component validates and stores the goal in the database, and how the Point System later uses these stored targets during periodic evaluations of user performance.

The **Personal Goal Management Activity Diagram** describes the control flow for creating and configuring a personal goal.

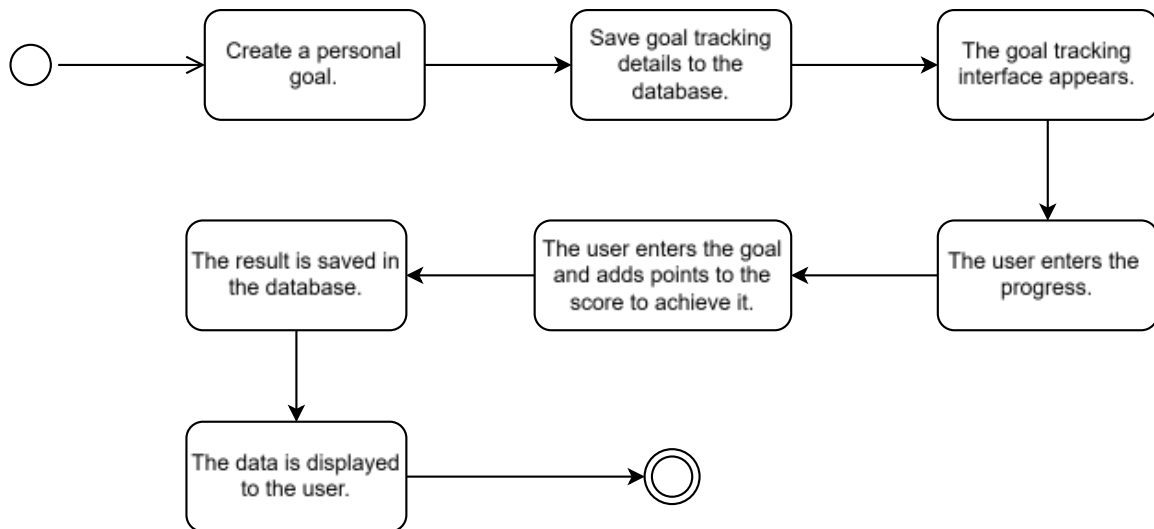
It covers entering the goal type and target value, defining the tracking duration and reminder settings, validating the configuration, storing the goal, and confirming successful creation to the user through the interface.

This diagram complements the tracking diagrams by explaining how targets are initially established before being used in daily evaluations.

5.4.5.1. Sequence Diagram:



5.4.5.2. Activity Diagram:



Chapter 6

Implementation

6.1. Chapter Introduction

Chapter 6 documents the implementation of the VitaMate system, translating the approved requirements and system design into a working mobile application and backend service. It explains how the Flutter client communicates with the Django REST API, how data is persisted, and where the core business logic (health metrics, goals, and rewards) is executed.

Because the project scope evolved during development, this chapter includes a scope freeze to clarify what is implemented in the current release, what is partially implemented, and what has been removed from scope. This improves traceability and sets clear expectations for the rest of the chapter.

6.1.1. Scope Freeze (Release Scope)

1. In Scope (Implemented in this release):

- Authentication (register, login, and token refresh using JWT).
- User profile management and health metric calculation (e.g., BMR/TDEE and daily goals).
- Nutrition tracking: meal logging and a food items database (FoodItem).
- Water intake logging and progress updates.
- Sleep logging and automatic sleep duration calculation.
- Steps tracking via the Pedometer sensor with Physical Activity permission handling.
- Dashboard/History summaries (calories, water, sleep, and steps progress).
- Local notifications on Android (e.g., water and activity reminders).

2. Out of Scope / Removed:

- GPS-based route and distance tracking (and any route visualization).
- Admin role and administrative features.

3. Partially Implemented:

- Sleep improvement feature (a flag/setting exists, but full guidance/logic is not implemented).
- BMI classification (BMI is computed, but category labeling/interpretation is not implemented).

- Downloadable/exportable reports (dashboard statistics exist, but no export/download is provided).
- iOS notifications verification (no execution evidence provided in this release).

The remainder of this chapter is organized as follows: the next section introduces the tools and technologies used during development. It then describes the backend structure, API routing, and key endpoints, followed by the frontend structure and its integration with the backend services.

6.2 Technology Stack and Libraries

This section summarizes the technologies, frameworks, and libraries used to implement VitaMate. The stack was selected to support a clean separation between the mobile client and backend services, provide secure authentication, enable sensor-based step tracking, and support local notifications. The following subsections describe the backend stack, frontend stack, and testing tools used in the current release.

6.2.1 Backend Technologies (Django / REST API)

The backend is implemented using **Django** with the **Django REST Framework (DRF)** to expose RESTful APIs consumed by the mobile application. Authentication is handled using **JSON Web Tokens (JWT)** via **SimpleJWT**, enabling stateless authentication with access/refresh token flows. During development, the project uses **SQLite** as the database for simplicity and rapid iteration.

Key backend technologies and libraries:

- **Django (v6)**: Core backend framework used for project structure, models, ORM, and configuration.
- **Django REST Framework (DRF)**: API layer (serializers, viewsets, permissions, routing).
- **SimpleJWT**: JWT-based authentication (access/refresh tokens, login/refresh endpoints).
- **django-cors-headers**: CORS configuration to support cross-origin requests during development/testing.
- **SQLite (development)**: Lightweight database used during development (with the possibility of switching to a production-grade DB later).

6.2.2 Frontend Technologies (Flutter Mobile Application)

The mobile application is implemented using **Flutter**, enabling a cross-platform user interface for Android and iOS. Network communication with the backend is performed using **Dio**, which supports interceptors for injecting authentication headers and handling token refresh flows. Authentication tokens are securely stored using **flutter_secure_storage**.

The application integrates device capabilities for activity tracking and reminders. Step tracking is implemented using the **pedometer** package and is guarded by runtime permission handling via **permission_handler**. Local notifications are scheduled using **flutter_local_notifications** together with the **timezone** package to ensure correct scheduling behavior across time zones. The UI and user experience are supported by additional libraries such as **intl** for formatting and **rive** for animations.

Key frontend technologies and libraries:

- **Flutter**: Cross-platform mobile development framework (UI, state management, navigation).
- **Dio**: HTTP client for API calls, supports interceptors (Bearer token injection and refresh on 401).
- **flutter_secure_storage**: Secure token storage using Keychain/Keystore.
- **shared_preferences**: Lightweight local persistence for non-sensitive preferences and flags.
- **flutter_local_notifications**: Local notification scheduling and display (Android implementation verified).
- **timezone**: Timezone-aware scheduling for notifications.
- **pedometer**: Step counting integration using device sensors.
- **permission_handler**: Runtime permission requests (Physical Activity, Notifications).
- **intl**: Date/time and number formatting utilities.
- **rive**: UI animations to improve user experience.

6.2.3 Development and Testing Tools

VitaMate includes automated tests on both the backend and frontend to validate core functionality and reduce regressions. The backend uses **Django TestCase** and **DRF APIClient** for endpoint-level testing. The frontend uses Flutter's built-in **flutter test** framework for unit and widget testing.

Testing tools:

- **Django TestCase + DRF APIClient**: Automated tests for authentication and selected API flows.
- **Flutter test**: Unit/widget tests for client-side logic (e.g., validation and networking behaviors).

6.2.4 Summary Table (Stack Overview)

Layer	Technologies / Libraries	Primary Purpose
<i>Backend</i>	Django, DRF, SimpleJWT, django-cors-headers	REST APIs, business logic, JWT authentication, CORS support
<i>Database (Dev)</i>	SQLite	Development-time persistence
<i>Frontend</i>	Flutter, Dio	Mobile UI + API integration
<i>Security (Client)</i>	flutter_secure_storage	Secure token storage
<i>Sensors/Permissions</i>	pedometer, permission_handler	Step tracking + runtime permissions
<i>Notifications</i>	flutter_local_notifications, timezone	Local reminders and scheduling
<i>Utilities/UI</i>	intl, shared_preferences, rive	Formatting, preferences, animations
<i>Testing</i>	Django TestCase, DRF APIClient, Flutter test	Backend & frontend automated tests

6.2.5 Table: Tools and Technologies Used in VitaMate

Tool / Technology	Category	Purpose
Git	Version Control	Managing source code versions and tracking changes during development.
GitHub	Collaboration Platform	Hosting the code repository and supporting team collaboration and version history.
Flutter	Frontend Framework	Developing a cross-platform mobile application using a single codebase.

Django	Backend Framework	Implementing backend logic and managing server-side operations.
Django REST Framework	API Framework	Building RESTful APIs for communication between frontend and backend.
RESTful API	Communication Standard	Enabling standardized and platform-independent data exchange via HTTP.
Visual Studio Code	Development Environment	Writing, debugging, and managing frontend and backend source code.
Postman	API Testing Tool	Testing, validating, and debugging API endpoints and responses.

6.2.6 Tools and Technologies Justification

The selected tools and technologies form an integrated development environment that supports modularity, scalability, and maintainability. The combination of Flutter and Django REST Framework enables a clear separation between frontend and backend layers while maintaining efficient communication through RESTful APIs. Version control and collaboration tools ensure structured development, while testing and development environments support code reliability and systematic verification. This toolset provides a stable foundation for implementing the system requirements and supporting future enhancements.

6.2.5 Versioning & Dependency Management

To ensure consistent builds and reproducible development environments, VitaMate manages its dependencies using standard tooling on both the backend and frontend. Backend Python packages are pinned to specific versions in **requirements.txt** and installed using `pip install -r requirements.txt`. Flutter dependencies are declared and resolved using **pubspec.yaml** and installed using `flutter pub get`.

- **Backend (Python/Django):** Dependency versions are pinned in **requirements.txt** (e.g., Django, DRF, SimpleJWT) to minimize version drift across development machines.

requirements

- **Frontend (Flutter):** Dependencies are maintained in **pubspec.yaml**, allowing reproducible builds using Flutter's package resolution tooling.

Backend dependency snapshot (from requirements.txt):

- **Django 6.0** (core backend framework)

requirements

- **djangorestframework 3.16.1** (REST API layer)

requirements

- **djangorestframework_simplejwt 5.5.1 + PyJWT 2.10.1** (JWT authentication)

requirements

- **django-cors-headers 4.9.0** (CORS configuration for development/testing)

requirements

- Supporting runtime packages: **asgiref, sqlparse, tzdata**

requirements

6.3 Backend Implementation

The VitaMate backend is implemented using Django and Django REST Framework (DRF) following a modular, app-based structure. It is responsible for authentication, data persistence, business logic execution, and exposing RESTful APIs to the Flutter mobile application. The backend codebase is organized into domain-focused Django apps to support separation of concerns and maintainability.

requirements

6.3.1 Project Structure (Backend Tree)

The following tree highlights the main backend files and directories (excluding local environment folders such as venv/ and runtime database artifacts):

VitaMate/

├─ manage.py

├─ requirements.txt

VitaMate English version

```
└─ vitamate_project/
|   └─ settings.py
|   └─ urls.py
|   └─ asgi.py
|   └─ wsgi.py
└─ users/
|   └─ models.py
|   └─ serializers.py
|   └─ views.py
|   └─ signals.py
|   └─ services/
|   └─ repositories/
|   └─ tests.py
|   └─ migrations/
└─ core/
|   └─ models.py
|   └─ serializers.py
|   └─ views.py
|   └─ services/
|   └─ repositories/
|   └─ tests.py
|   └─ migrations/
└─ gamification/
|   └─ models.py
|   └─ points_service.py
|   └─ repositories/
|   └─ tests.py
|   └─ migrations/
└─ loadtest/
└─ locustfile.py
```

requirements

6.3.2 Key Entry Points and Configuration

manage.py: Django entry point used to run the server, apply migrations, and run tests.

requirements

requirements.txt: pinned backend dependencies (Django, DRF, SimpleJWT, CORS, etc.).

VitaMate English version

vitamate_project/settings.py: global configuration including installed apps, REST framework settings, JWT configuration, CORS, middleware, and database.

vitamate_project/urls.py: API routing (authentication endpoints + DRF routers + dashboard/history routes).

requirements

asgi.py / wsgi.py: backend entry points for deployment.

requirements

6.3.3 Backend Layering (How Responsibilities Are Separated)

To keep the implementation maintainable and testable, VitaMate separates responsibilities across multiple layers:

Models (models.py): database entities and computed fields (e.g., logs, derived values).

Serializers (serializers.py): validation and JSON transformation; controls read-only/computed fields.

Views (views.py): DRF ViewSets/APIViews that expose endpoints and enforce access permissions.

Services (services/): business logic and calculations (e.g., goals, aggregation, scoring hooks).

Repositories (repositories/): data access patterns and query encapsulation (e.g., daily upsert logic).

Tests (tests.py): API-level tests and critical functional checks.

This structure supports clean separation of concerns and simplifies future extensions.

requirements

6.3.4 Modules Overview

1) users/ (Identity & Profile Layer)

Handles registration and user profile management.

signals.py initializes related records upon account creation (e.g., profile/score initialization as designed).

Services and repositories encapsulate profile updates and derived targets.

requirements

2) core/ (Health Tracking Domain Layer)

Contains the main tracking entities and APIs, including: FoodItem, MealLog, WaterLog, ActivityLog, StepLog, SleepLog (and additional entities such as Medicine/Habit if included in the release).

Provides aggregated endpoints such as DashboardView and StatsHistoryView to compute daily and historical summaries.

requirements

3) gamification/ (Points & Rewards Layer)

Stores user score data (e.g., points, level) and applies reward/penalty logic through points_service.py.

Designed to be triggered by tracking actions (e.g., logging water/sleep/activity).

requirements

4) loadtest/ (Performance Evidence)

Includes locustfile.py defining a load scenario (e.g., login + dashboard + logging flows) to support NFR performance verification when executed.

requirements

6.3.5 API Routing Summary (Where Endpoints Are Defined)

Routing is centralized in vitamate_project/urls.py and consists of:

Authentication endpoints (JWT-based):

/api/auth/register/, /api/auth/login/, /api/auth/refresh/, and profile endpoint (e.g., /api/auth/me/) as implemented.

requirements

Core API endpoints (router-based):

/api/meals/, /api/water/, /api/steps/, /api/activities/, /api/sleep/, /api/habits/ (as available in the current code).

requirements

Aggregation endpoints:

/api/dashboard/ for daily summaries and progress indicators.

requirements

6.3.6 Traceability Summary (Features → Code Locations)

This mapping summarizes where the major implemented features are handled in the backend:

Authentication (FR-01): users/views.py, users/serializers.py, and auth routes in vitamate_project/urls.py.

requirements

Profile & Health Data (FR-02): users/models.py (UserProfile), profile update serializers/views, and related services/repositories.

requirements

Nutrition & Calories (FR-16–FR-20): core/models.py (FoodItem/MealLog), core/views.py (Meal endpoints), and nutrition services/repositories for aggregation.

requirements

Water Tracking (FR-20–FR-26): core/models.py (WaterLog), core/views.py, and water services/repositories.

requirements

Steps Tracking (FR-07–FR-08): core/models.py (StepLog + constraints), step repositories (daily upsert pattern), and step services (distance estimation if designed).

requirements

Activity Logging & Burned Calories (FR-09–FR-11/FR-10): core/models.py (ActivityLog), core/views.py, and activity services.

requirements

Sleep Tracking (FR-28–FR-29): core/models.py (SleepLog + duration computation), core/views.py, and sleep services/repositories.

requirements

Dashboard/Progress (FR-32/FR-35): core/views.py (DashboardView/StatsHistoryView) returning aggregated progress and indicators.

requirements

Note: Notification scheduling is implemented on the client side (Flutter) in this release, while the backend provides stored data and summaries consumed by the dashboard and history views.

Requirements

6.3.3 Component Summary Table (Backend)

Component	Type	Description
<i>manage.py</i>	File	Entry point for running server, migrations, and tests.
<i>requirements.txt</i>	File	Pinned backend dependencies (Django/DRF/SimpleJWT/CORS).
<i>vitamate_project/</i>	Directory	Global configuration and system-level settings.
<i>settings.py</i>	File	INSTALLED_APPS, DB settings, DRF config, JWT, middleware, CORS.
<i>urls.py</i>	File	API routing (auth + router endpoints + dashboard/history routes).
<i>users/</i>	App	Authentication + user profile management and derived targets.
<i>core/</i>	App	Main health domain (meals, water, steps, sleep, activities, dashboard).
<i>gamification/</i>	App	Points and levels logic via points_service.
<i>loadtest/</i>	Module	Load scenario (Locust) for NFR evidence when executed.

Module Responsibilities (Formal Description)

1) *vitamate_project/* (Project configuration layer)

- *settings.py*: Global configuration (installed apps, database settings, middleware, authentication).
- *urls.py*: API routing; registers DRF routers and authentication endpoints.
- *asgi.py* / *wsgi.py*: Deployment entry points.

2) users/ (Identity and user profile layer)

- views.py: Registration endpoint (RegisterView).
- serializers.py: User registration validation (RegisterSerializer).
- models.py: UserProfile stores user health-related baseline data and targets (e.g., water, calories, activity goal).
- signals.py: Creates default UserProfile and initializes UserScore when a new user is created.

3) core/ (Health tracking domain layer)

Contains most tracking entities and APIs:

- Models: FoodItem, MealLog, WaterLog, Exercise, ActivityLog, StepLog, SleepLog, Medicine, MedicineLog, Habit, HabitLog.
- ViewSets: CRUD endpoints for daily tracking (nutrition, water, activity, sleep, habits).
- DashboardView: Aggregates daily summaries (calories, hydration, steps/distance, points).

4) gamification/ (Reward and points layer)

- models.py: UserScore maintains points and level progression.
- Points are updated from tracking actions (e.g., water logging adds points).

API Routing (Where endpoints are defined)

Routing is defined in: vitamate_project/urls.py

- Authentication:
 - api/auth/register/ → users.views.RegisterView
 - api/auth/login/ → JWT token obtain (SimpleJWT)
 - api/auth/refresh/ → JWT refresh
- Core APIs (router under api/):
 - /api/meals/, /api/water/, /api/medicines/, /api/steps/, /api/activities/, /api/sleep/, /api/habits/
- Aggregation:

- /api/dashboard/ → core.views.DashboardView

6.3.4 Layering (How Responsibilities Are Separated)

To keep the backend maintainable, responsibilities are separated into layers: Models (data entities), Serializers (validation + transformation), Views (API endpoints), Services (business logic), Repositories (data access patterns), and Tests (verification). This structure reduces coupling and supports scalable evolution of features.

Traceability: Sprints & Requirements → Code Locations

Sprint 1 — Core setup & user management

Backlog items: PB-06, PB-07, PB-08, PB-09, PB-10, PB-11

- **Account creation (FR-01)**

- users/views.py → RegisterView
- users/serializers.py → RegisterSerializer
- vitamate_project/urls.py → /api/auth/register/

- **Login / secure auth (JWT access/refresh)**

- vitamate_project/urls.py → /api/auth/login/, /api/auth/refresh/ (SimpleJWT)

- **Profile baseline creation (FR-02: health baseline entry — server-side initialization)**

- users/signals.py → auto-creates UserProfile with defaults on user creation
- users/models.py → UserProfile fields (height, weight, gender, activity level, goal, etc.)

- **Profile retrieval/update endpoint (FR-02: profile update / “me”)**

- users/views.py → ManageUserView
- users/serializers.py → user/profile update serializer
- vitamate_project/urls.py → /api/auth/me/

- **Initial DB structure**

- users/migrations/, core/migrations/, gamification/migrations/

Not found in backend:

- **Account deletion endpoint (FR-XX / if required) — not implemented server-side**

Sprint 2 — Daily & health tracking

Backlog items: PB-12, PB-13, PB-14, PB-15, PB-16, PB-17

- **Sleep tracking (FR-28/FR-29)**

- Model: core/models.py → SleepLog (+ duration_hours computation/property)

- o API: core/views.py → SleepLogViewSet

- o Route: vitamate_project/urls.py → /api/sleep/

- **Water intake logging (FR-25) + motivational points (FR-26)**

- o Model: core/models.py → WaterLog

- o API: core/views.py → WaterLogViewSet

- o Points: gamification/points_service.py → award water points on logging action (e.g., fixed points rule)

- o Route: /api/water/

- **Nutrition & calories (FR-17/FR-18/FR-19/FR-20)**

- o Models: core/models.py → FoodItem, MealLog

- o API: core/views.py → MealLogViewSet (+ FoodItem endpoints if exposed)

- o Aggregation: core/services/nutrition_service.py → daily consumed calories aggregation

- o Routes: /api/meals/ (and /api/food-items/ if enabled)

- **Steps tracking (FR-07) + distance estimation (no GPS routes)**

- o Model: core/models.py → StepLog (unique per user/date)

- o Repo: core/repositories/step_log_repository.py → daily upsert (one log/day)

- o Service: core/services/steps_service.py → distance estimation logic (when required)

- o API: core/views.py → StepLogViewSet

- o Route: /api/steps/

- **Statistics visualization / dashboard (FR-35 / progress summaries)**

- o API: core/views.py → DashboardView + StatsHistoryView

- o Routes: /api/dashboard/ (+ /api/history/ or similar stats/history route if exposed)

Not found in backend:

- Notifications/reminders scheduling (FR-13/FR-24/FR-34) — no scheduling service in backend code (handled client-side in Flutter via local notifications)

Sprint 3 — Activity tracking & sensor-based features

Backlog items: PB-18, PB-19, PB-20, PB-21, PB-22

- **PB-18 (GPS run tracking) — Removed / Out of Scope in this release (no GPS route/session tracking server-side)**

- **Workout/activity logging (FR-09)**

- o Model: core/models.py → ActivityLog

- o API: core/views.py → ActivityLogViewSet

- o Route: /api/activities/

- **Calories burned estimation (FR-11)**

- o Logic: core/models.py → ActivityLog.calories_burned (MET formula referencing user weight from UserProfile)

- o Service (if used): core/services/activity_service.py

- **Sensor integration (PB-21)**

- o Backend: receives step/activity logs via API; sensor reading itself is client-side

- o Frontend evidence: Flutter uses pedometer + permission_handler (not implemented server-side)

- **Dashboard updates (PB-22)**

- o Backend: DashboardView aggregates daily metrics for the frontend display

Sprint 4 — Goals & motivation

Backlog items: PB-23, PB-24, PB-25, PB-26, PB-27

- **User targets / goal-related baselines (FR-05/FR-15/FR-16/FR-22/FR-08)**

- o users/models.py / profile computation logic → derived targets (daily calories, water target, step goal, burn goal, etc.)

- o Service: users/user_profile_service.py → calculations and target updates (BMR/TDEE, steps goal, water goal)

- **Habits tracking (FR-39 — if included in current release)**

- o Models: core/models.py → Habit, HabitLog (and Medicine if used)

- o APIs: core/views.py → HabitViewSet (+ HabitLogViewSet if exposed)

- o Route: /api/habits/ (and related habit logs route if configured)

- **Reward system (FR-31/FR-32)**

- o gamification/models.py → UserScore (total_points, level)

- o gamification/points_service.py → reward/penalty rules

- o Initialized: users/signals.py and updated via tracking actions (e.g., water/steps/sleep/activity logging)

Not found in backend:

- A dedicated Goal entity/model and a unified goal reminders configuration module — targets are computed from profile fields; reminders are client-side

Sprint 5 — Testing, refinement & delivery

Backlog items: PB-28, PB-29, PB-30, PB-31

- **core/tests.py, users/tests.py, gamification/tests.py** exist.
- **Backend tests include functional checks for key flows (e.g., auth and selected logging/isolation behaviors), while some areas remain pending for extended coverage (recommended to expand TC-to-test mapping).**
- **Load testing support is available via loadtest/locustfile.py (20-user scenario: login + dashboard + common logging flows) if executed as evidence.**

API Endpoints Table (Backend – Django REST Framework)

Note: ViewSets registered in the backend router expose standard REST operations:

- **Collection:** GET /api/<resource>/ (list), POST /api/<resource>/ (create)
- **Detail:** GET/PUT/PATCH/DELETE /api/<resource>/<id>/

Authentication

Endpoint	Method	Purpose
/api/auth/register/	POST	Create a new user account.
/api/auth/login/	POST	Obtain JWT access/refresh tokens.
/api/auth/refresh/	POST	Refresh JWT access token.

Core System APIs

Module	Endpoint	Methods	Purpose
Dashboard	/api/dashboard/	GET	Return aggregated daily summary and progress overview.
Nutrition	/api/meals/	GET, POST	List or create meal logs (calories tracking).
Water	/api/water/	GET, POST	List or create water intake logs.
Steps	/api/steps/	GET, POST	List or create step logs (steps and derived distance if applicable).

Sleep	/api/sleep/	GET, POST	List or create sleep logs and compute duration.
Activities	/api/activities/	GET, POST	List or create activity logs (duration/calories burned).

6.4 Frontend Structure (Flutter)

Introduction

The VitaMate mobile application is implemented using **Flutter** and organized using a **feature-based modular structure**. The codebase separates authentication, shared infrastructure, and domain features (home/dashboard, steps, water, sleep, nutrition, activity, statistics) to improve maintainability and reduce coupling between modules. Each feature typically follows a consistent internal layout: **data (API)**, **models**, **state/controllers**, and **screens/UI**, while reusable components and utilities are placed under `shared/`.

In this release, **notifications scheduling and sensor integration are handled on the client side** (Flutter). The backend provides REST endpoints for persistence and aggregation (e.g., dashboard summaries and history).

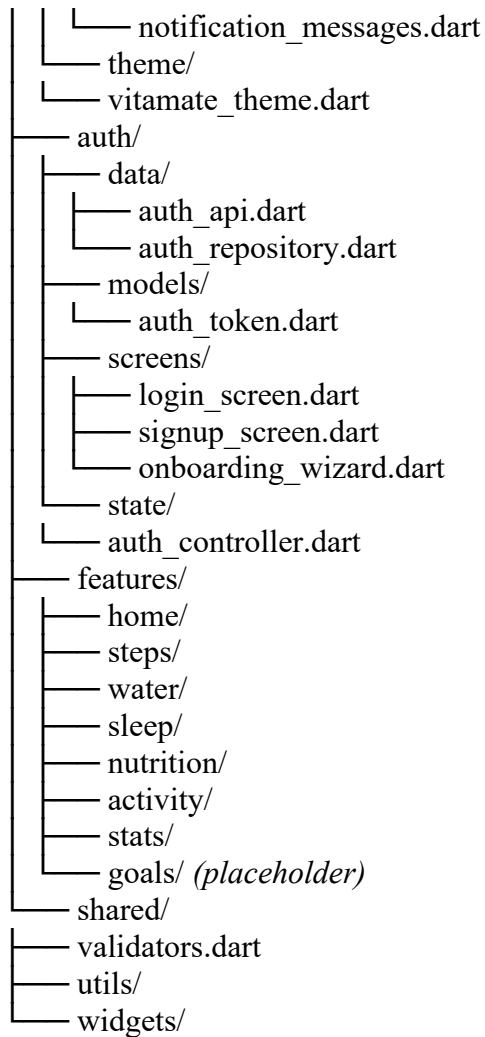
Note: Some modules (e.g., goals) exist as placeholders and are not fully implemented in the current release.

Project Tree (Flutter – lib/)

```

lib/
├── main.dart
├── app.dart
├── core/
│   ├── api_endpoints.dart
│   ├── network/
│   │   ├── http_client.dart
│   │   └── auth_interceptor.dart
│   ├── routing/
│   │   └── routes.dart
│   ├── storage/
│   │   └── secure_storage.dart
│   ├── notifications/
│   │   ├── notifications_service.dart
│   │   ├── notification_channels.dart
│   │   └── notification_ids.dart

```



Structure Explanation (Table)

Component	Description
<i>pubspec.yaml</i>	Declares Flutter dependencies (e.g., Dio, secure storage, local notifications, timezone, pedometer, permission handler, intl, rive) and defines assets (avatars/images).
<i>main.dart</i>	Application entry point. Initializes required services (e.g., notifications/timezone if used) and launches the app using runApp.
<i>app.dart</i>	Root widget. Configures MaterialApp, theme, initial route (e.g., Login), and registers route mappings for screens.

<i>core/</i>	Cross-cutting infrastructure used by all modules: API endpoints, networking, storage, routing, theme, and notifications.
<i>core/api_endpoints.dart</i>	Centralizes baseUrl and REST paths (auth, dashboard/history, meals, water, steps, activities, sleep, habits, foods, etc.). This avoids scattering URLs across features.
<i>core/network/http_client.dart</i>	Creates the Dio HTTP client with timeouts and logging, and attaches the authentication interceptor.
<i>core/network/auth_interceptor.dart</i>	Injects Authorization: Bearer <access_token> into requests and triggers a refresh flow on HTTP 401 using the refresh token. This supports secure, stateless sessions.
<i>core/storage/secure_storage.dart</i>	Secure persistence for JWT tokens using Keychain/Keystore. Tokens are not stored in plain preferences.
<i>core/routing/routes.dart</i>	Defines route names and navigation structure used by the app.
<i>core/theme/vitamate_theme.dart</i>	Defines a unified theme (colors/typography) to ensure consistent UI styling across screens.
<i>core/notifications/notifications_service.dart</i>	Client-side scheduling/canceling of local notifications (water, sleep bedtime/wake, meals, activity, steps). Includes Android permission checks and exact alarm handling.
<i>core/notifications/*</i>	Notification constants: channel definitions, IDs, and message templates to keep notification logic consistent and maintainable.
<i>auth/</i>	Authentication module containing screens, API calls, token model, repository, and controller/state.
<i>auth/data/auth_api.dart</i>	Performs register/login and user profile retrieval/update (me, updateMe) via Dio.
<i>auth/data/auth_repository.dart</i>	Encapsulates auth workflows (save/read tokens, logout, call getMe/updateMe). Keeps token handling out of UI screens.

auth/models/auth_token.dart

Defines token structures used across the auth layer (access/refresh).

auth/state/auth_controller.dart

Controls authentication state (logged-in/out), coordinates repository calls, and triggers navigation transitions.

features/

Domain features grouped by module. Each module typically includes: data/ (API), models/, state/ (controllers), and screens/ (UI).

features/home/

Fetches dashboard/history data and builds dashboard_data.dart summaries. Provides the main home screen and navigation cards.

features/steps/

Manages pedometer stream, baseline handling, permission requests (Activity Recognition), server sync for step logs, and step reminder UI.

features/water/

Logs water intake, fetches progress from dashboard, and provides UI controls (tiles/buttons) for quick entry.

features/sleep/

Handles sleep logs, sleep settings, and reminders. Includes controllers for logging and profile-based settings updates.

features/nutrition/

Manages food items + meal logging and calorie aggregation into daily summaries (consumed/remaining).

features/activity/

Handles exercise listing and activity logging, with calories burned estimation and optional reminders.

features/stats/

Loads multi-day history (e.g., 7 days) and renders trend summaries in the stats screen.

features/goals/ (placeholder)

Present as basic scaffolding (API/model/controller/screen), but not fully implemented as a complete goal management system in this release.

shared/validators.dart

Input validation utilities (email/password and basic constraints).

shared/widgets/

Reusable UI components (e.g., app bar, buttons, labeled rows, progress header) shared across features.

<i>widget_test.dart</i>	Smoke test verifying the app launches and reaches the expected initial screen (e.g., Login).
<i>assets/</i>	Static resources (avatars/images) referenced in pubspec.yaml.
<i>platform folders</i>	Android configuration includes required permissions (notifications, exact alarms, boot completed, activity recognition, internet). Other platforms keep the default Flutter structure unless extended.

State Management Approach (Implementation Note)

VitaMate manages client-side state using **feature controllers** (e.g., *_controller.dart) that coordinate API calls through their corresponding API/repository layer and expose UI-ready state to screens. This approach keeps networking and business logic outside UI widgets, improves testability, and reduces coupling between UI and data access.

Security Boundary (Client-Side)

Authentication tokens are stored using **secure storage** (Keychain/Keystore) and are not stored in plain preferences. The networking layer uses a Dio **authentication interceptor** to attach Bearer tokens to requests and automatically attempt token refresh on 401 responses. This design supports secure, stateless sessions while keeping sensitive token handling centralized and consistent.

Client vs Server Responsibilities (Clarification)

- **Frontend (Flutter):** sensor reading (pedometer stream), runtime permission requests, and local notification scheduling/canceling.
- **Backend (Django/DRF):** persistence of logs (meals, water, steps, sleep, activities) and aggregation endpoints (dashboard/history).
This division matches the current release scope, where reminders are implemented as **local notifications** rather than server-push scheduling.

Build & Run (Client Reproducibility)

To run the client application:

- Install dependencies: `flutter pub get`
- Launch the app: `flutter run`
The backend baseUrl and endpoint paths are centralized in `core/api_endpoints.dart`.
Android permissions required for notifications and activity recognition are declared in `AndroidManifest.xml`.

Evidence (Where Verification Is Captured)

Frontend verification evidence for this release can be demonstrated through:

- Successful login and token persistence (secure storage).
- Automatic refresh flow on 401 (interceptor behavior).
- Step permission flow and step values updating via the pedometer stream.
- Scheduled notifications appearing on Android (water/activity/sleep reminders).
These artifacts can be referenced in the Testing & Evidence section of this chapter.

6.6 Testing & Verification

6.6.1 Purpose and Scope of Testing

This section documents the testing and verification activities conducted to validate VitaMate against its functional requirements (FRs) and selected non-functional requirements (NFRs). Verification was performed across both the backend (Django/DRF APIs) and the frontend (Flutter mobile application), covering authentication, daily health logging (water, steps, sleep), dashboard aggregation, and client-side platform behavior (secure storage, permissions, and notifications).

The verification scope follows the project scope-freeze decisions: features removed from scope (e.g., GPS tracking and Admin functionality) are not included in the verification matrix. The objective is to provide reproducible evidence that the implemented scope works correctly and reliably under normal usage conditions.

6.6.2 Test Strategy and Levels

Testing was designed using a layered approach to provide evidence at multiple levels:

Backend API Tests (Django/DRF)

Backend tests validate API correctness, authentication enforcement, and basic functional flows. These tests focus on server-side correctness because it is the authoritative layer for user data integrity and security controls.

Frontend Widget/Unit Tests (Flutter)

Flutter tests validate client-side behavior including field validation, navigation after authentication, secure token storage operations, permissions UX for step tracking, and notification scheduling. Platform integrations are mocked (MethodChannels and fake network adapters) to ensure deterministic tests without requiring emulators or a live backend.

Load/Performance Testing (Locust)

Load testing is used to assess performance and concurrency under “normal conditions” by simulating concurrent users performing realistic tasks (auth + dashboard + daily logs). This level provides quantifiable metrics (average latency, p95 latency, failure rate, and throughput).

6.6.3 Test Environment and Reproducibility

Tests were executed locally using the following reproducible commands and environments:

Backend (Django/DRF)

- Command: `python manage.py test`
- Environment: local development setup using a virtual environment
- Evidence captured as a console output screenshot.

Evidence:

- **EV-01 (Backend Tests Output):**

```
(.venv) PS C:\Users\Salam Ayash\Desktop\VitaMate\Implementation\vitamate_backend> python manage.py test
Found 5 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 5 tests in 2.729s

OK
Destroying test database for alias 'default'...
```

(Place in this subsection; shows backend tests execution and “OK” result.)

Note (documented limitation): The current captured run reports “Found 5 test(s)”, indicating that the evidence reflects a partial test discovery/execution subset in the current environment.

Frontend (Flutter)

- Command: flutter test
- Environment: tests executed locally with mocked platform channels and a fake Dio adapter.

Evidence:

- **EV-02 (Flutter Tests Output):**

```
PS C:\Users\Salam Ayash\Desktop\VitaMate\Implementation\vitamate_frontend> flutter test
Content-Type: application/json
data:
{username: user1, password: Secret123}

*** Response ***
uri: http://10.170.235.91:8000/api/auth/login/
statusCode: 200
headers:
  content-type: application/json
Response Text:
{"access":"tok","refresh":"rtok"}

00:02 +5: All tests passed!
```

(Place here; shows “All tests passed”).

Load Testing (Locust)

- Command:
locust -f loadtest/locustfile.py --host=http://127.0.0.1:8000 --users 20 --spawn-rate 5
- Scenario: 20 concurrent virtual users, 1–3 seconds wait time, bounded user pool, and tasks hitting auth/dashboard/logging endpoints.

Evidence:

- EV-03A (Locust Statistics Screen):

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/auth/login/	20	0	1900	2800	2800	2113.78	1252	2827	491	0	0
GET	/api/auth/me/	20	0	59	230	230	80.82	14	230	176.5	0	0
POST	/api/auth/register/	20	0	2100	3300	3300	2487.94	1615	3345	94	0	0
GET	/api/dashboard/	279	0	23	110	220	38.32	10	474	469.87	4	0
POST	/api/sleep/	88	0	32	86	170	40.76	14	173	178.94	1	0
POST	/api/steps/	188	0	38	110	170	47.38	21	180	126	2.4	0
POST	/api/water/	190	0	36	120	190	47.88	20	302	58.69	2.6	0
	Aggregated	805	0	35	470	2900	166.44	10	3345	244.84	10	0

- EV-03B (Locust Percentiles Table):

Response time percentiles (approximated)													
Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	
reqs													
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----
POST 20	/api/auth/login/	2400	2600	2700	2700	2800	2800	2800	2800	2800	2800	2800	
GET 20	/api/auth/me/	59	110	120	130	170	230	230	230	230	230	230	
POST 20	/api/auth/register/	2800	3000	3200	3200	3300	3300	3300	3300	3300	3300	3300	
GET 667	/api/dashboard/	22	28	35	41	71	90	110	140	470	470	470	
POST 225	/api/sleep/	31	36	41	45	62	81	100	170	170	170	170	
POST 438	/api/steps/	37	42	51	57	77	100	150	170	180	180	180	
POST 440	/api/water/	35	39	44	50	63	84	130	180	300	300	300	
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----
-----	Aggregated	32	38	45	52	79	110	1400	2400	3300	3300	3300	
1830													

- EV-03C (Locust Summary Output/Run Evidence):

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
POST	/api/auth/login/	20	0(0.00%)	2113	1252	2827	1900	0.11	0.00
GET	/api/auth/me/	20	0(0.00%)	80	13	230	59	0.11	0.00
POST	/api/auth/register/	20	0(0.00%)	2487	1614	3344	2100	0.11	0.00
GET	/api/dashboard/	667	0(0.00%)	33	6	474	22	3.58	0.00
POST	/api/sleep/	225	0(0.00%)	37	10	172	31	1.21	0.00
POST	/api/steps/	438	0(0.00%)	45	14	183	37	2.35	0.00
POST	/api/water/	440	0(0.00%)	41	12	302	35	2.36	0.00
Aggregated		1830	0(0.00%)	88	6	3344	32	9.81	0.00

- EV-03D (Locust Run Start/Stop + Summary):

```
(.venv) PS C:\Users\Salam Ayash\Desktop\VitaMate\Implementation\vitamate_backend> locust -f loadtest/locustfile.py --host=http://127.0.0.1:8000 --users 20 --spawn-rate 5
Traceback (most recent call last):
  File "C:\Users\Salam Ayash\Desktop\VitaMate\Implementation\vitamate_backend\.venv\lib\site-packages\gevent\_ffi\loop.py", line 100, in python_check_callback
    def python_check_callback(self, watcher_ptr): # pylint:disable=unused-argument
KeyboardInterrupt
2026-01-27T04:45:09Z
[2026-01-27 07:45:09,307] DESKTOP-GSL1AU1/INFO/locust.main: Shutting down (exit code 0)
Type      Name      # reqs      # fails      Avg      Min      Max      Med      req/s      failures/s
-----
POST      /api/auth/login/      20      0(0.00%)      2113      1252      2827      1900      0.11      0.00
GET        /api/auth/me/          20      0(0.00%)          80         13        230         59      0.11      0.00
POST      /api/auth/register/    20      0(0.00%)      2487      1614      3344      2100      0.11      0.00
GET        /api/dashboard/       667      0(0.00%)         33          6        474         22      3.58      0.00
POST      /api/sleep/           225      0(0.00%)         37         10        172         31      1.21      0.00
POST      /api/steps/           438      0(0.00%)         45         14        183         37      2.35      0.00
POST      /api/water/           440      0(0.00%)         41         12        302         35      2.36      0.00
-----
Aggregated                                1830      0(0.00%)         88          6        3344         32      9.81      0.00
```

(Place all four in this subsection or in Appendix A; reference them here.)

6.6.4 Backend Verification (API Tests)

6.6.4.1 Coverage

Backend testing validates:

- API accessibility and correctness under authentication
- Core correctness for implemented endpoints in the backend layer
- Server-side enforcement of authentication and request handling

(Given the captured evidence run is partial, the verification evidence confirms successful execution of the backend test runner and successful completion of the discovered tests.)

6.6.4.2 Test Suite Structure (As Implemented)

The backend test suite is organized into domain-specific test modules (authentication, water, steps, sleep, nutrition, activity, isolation, and points) and supported by shared test helpers/factories. This structure enables scalable additions while maintaining clarity and reducing duplication.

6.6.4.3 Execution Results

The backend tests were executed successfully and completed with an “OK” status in the captured run.

Evidence:

```
(.venv) PS C:\Users\Salam Ayash\Desktop\VitaMate\Implementation\vitamate_backend> python manage.py test
Found 5 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 5 tests in 2.729s

OK
Destroying test database for alias 'default'...
```

6.6.5 Frontend Verification (Flutter Tests)

6.6.5.1 Coverage

Frontend tests verify critical client behaviors through deterministic widget/unit tests:

- Authentication validation and navigation behavior
- Secure token persistence through mocked secure storage channels
- Permission handling UX (steps tracking)
- Notifications scheduling calls using mocked notification channels
- Smoke test to confirm correct initial screen

6.6.5.2 Rationale for Mocks

Mocks were used to eliminate dependency on:

- live backend connectivity
- device/emulator hardware sensors
- platform-specific services (notifications, permissions, secure storage)

This ensures reproducibility and consistent results in automated test runs.

6.6.5.3 Execution Results

Flutter tests executed successfully, and the captured run indicates all tests passed.

Evidence:

```
PS C:\Users\Salam Ayash\Desktop\VitaMate\Implementation\vitamate_frontend> flutter test
Content-Type: application/json
data:
{username: user1, password: Secret123}

*** Response ***
uri: http://10.170.235.91:8000/api/auth/login/
statusCode: 200
headers:
  content-type: application/json
Response Text:
{"access":"tok","refresh":"rtok"}

00:02 +5: All tests passed!
```

6.6.6 Performance & Concurrency Verification (Locust)

6.6.6.1 Scenario Definition (“Normal Conditions”)

The load test simulates **20 concurrent users** with a **spawn rate of 5** and **1–3 seconds wait time** between actions. Users perform a realistic sequence:

- authentication (register/login) with a bounded user pool to prevent unbounded registrations
- GET /api/auth/me/
- GET /api/dashboard/
- POST daily logs: /api/water/, /api/steps/, /api/sleep/

6.6.6.2 Metrics Collected

The following metrics were collected:

- average latency per endpoint
- p95 latency per endpoint
- failure rate (errors/timeouts)
- throughput (requests per second)

6.6.6.3 Results (from captured evidence)

The test ran with **20 users** and achieved **0% failures**. Core daily endpoints (dashboard and logging operations) showed low latencies, while authentication endpoints were slower under the local development environment (expected due to heavier auth and user creation operations).

Results Table (from EV-03A)

Endpoint	Method	Avg (ms)	p95 (ms)	Fail %
/api/auth/register/	POST	2487.94	3300	0
/api/auth/login/	POST	2113.78	2800	0
/api/auth/me/	GET	80.82	230	0
/api/dashboard/	GET	38.32	110	0
/api/water/	POST	47.88	120	0
/api/steps/	POST	47.38	110	0
/api/sleep/	POST	40.76	86	0
Aggregated	—	156.44	470	0

Evidence placement:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/auth/login/	23	0	1800	2800	2800	2113.78	1252	2827	491	0	0
GET	/api/auth/me/	23	0	59	230	230	80.82	14	230	175.5	0	0
POST	/api/auth/register/	23	0	2130	3300	3300	2487.94	1818	3345	94	0	0
GET	/api/dashboard/	278	0	23	110	220	38.32	10	474	469.87	4	0
POST	/api/sleep/	88	0	32	86	170	40.76	14	173	178.94	1	0
POST	/api/steps/	180	0	38	110	170	47.38	21	180	120	2.4	0
POST	/api/water/	180	0	38	120	180	47.88	20	302	59.69	2.8	0
POST	Aggregated	816	0	35	470	2500	156.44	10	3345	244.94	10	0

Response time percentiles (approximated)													
Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	#
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----
POST	/api/auth/login/	2400	2600	2700	2700	2800	2800	2800	2800	2800	2800	2800	28
GET	/api/auth/me/	59	110	120	130	170	230	230	230	230	230	230	
POST	/api/auth/register/	2800	3000	3200	3200	3300	3300	3300	3300	3300	3300	3300	20
GET	/api/dashboard/	22	28	35	41	71	90	110	140	470	470	470	
POST	/api/sleep/	31	36	41	45	62	81	100	170	170	170	170	225
POST	/api/steps/	37	42	51	57	77	100	150	170	180	180	180	
POST	/api/water/	35	39	44	50	63	84	130	180	300	300	300	438
Aggregated		32	38	45	52	79	110	1400	2400	3300	3300	3300	
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----
1830													

6.6.7 Traceability Matrix (FR/NFR → Tests → Evidence)

6.6.7.1 FR Traceability (Evidence-backed)

Functional Capability	Verification Method	Evidence
<i>Backend functional correctness (discovered tests run)</i>	Django test runner	EV-01
<i>Frontend authentication flow + validation</i>	Flutter widget/unit tests	EV-02
<i>Core daily usage operations (dashboard + water/steps/sleep APIs under load)</i>	Locust scenario	EV-03A, EV-03B
<i>User-facing flows (visual confirmation)</i>	UI screenshots	EV-UI-*

6.6.7.2 NFR Traceability (Evidence-backed)

NFR	Verification Method	Evidence
<i>NFR-01 Performance</i>	Locust avg/p95 latency	EV-03A, EV-03B
<i>NFR-02 Concurrency</i>	Locust 20 users + 0% failures	EV-03A
<i>NFR-08 Usability (supporting evidence)</i>	UI screenshots of main flows	EV-UI-*

6.6.8 UI Evidence (Screenshots of the Application)

To complement automated tests, UI screenshots were captured to visually confirm that the implemented scope is accessible and functional from the user's perspective. The screenshots cover login, main dashboard/home navigation, and core feature screens (water, steps, sleep, nutrition, activity).

Evidence (place in Appendix A – UI Evidence):

- **EV-UI-01:**

8:04

VitaMate

*Begin your journey to better health
with your smart health assistant.*



Username

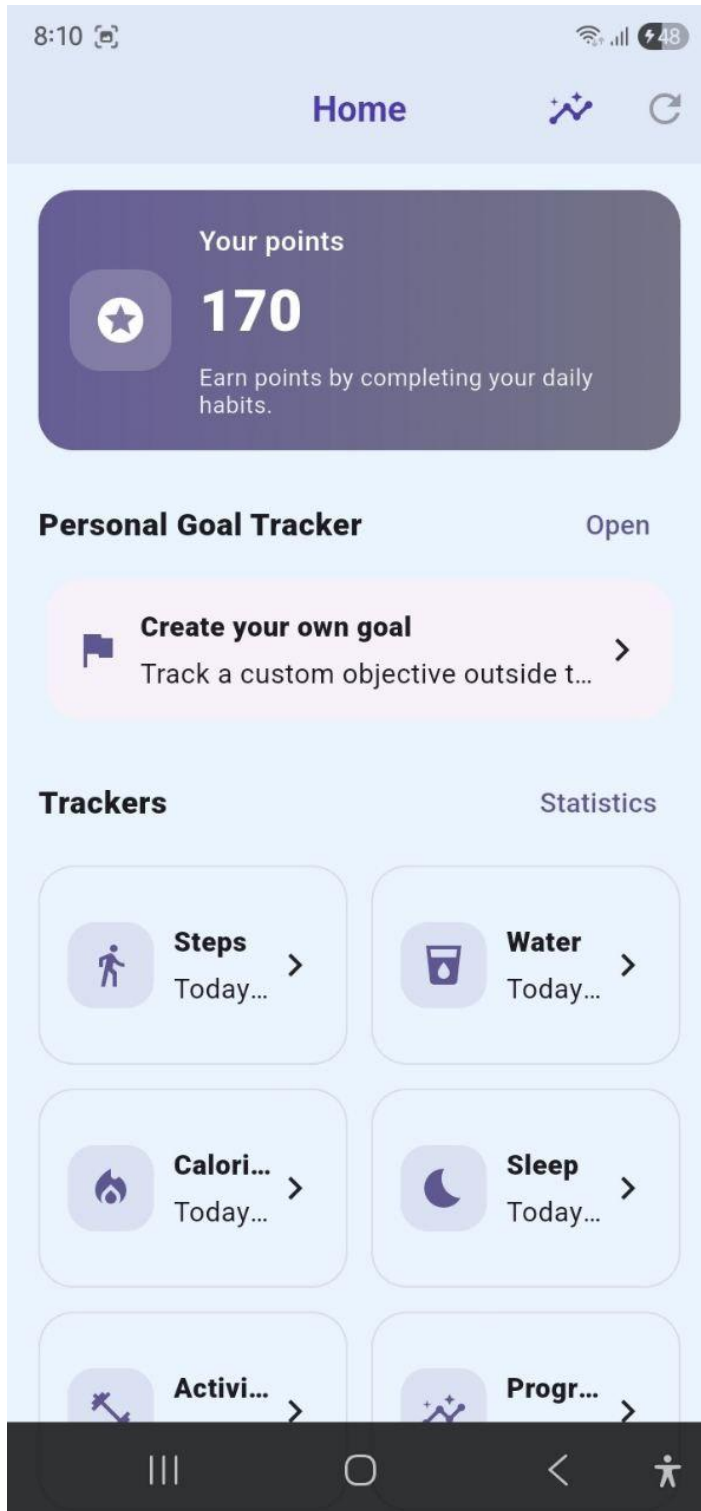
Password 

Login

[Don't have an account? Sign up](#)

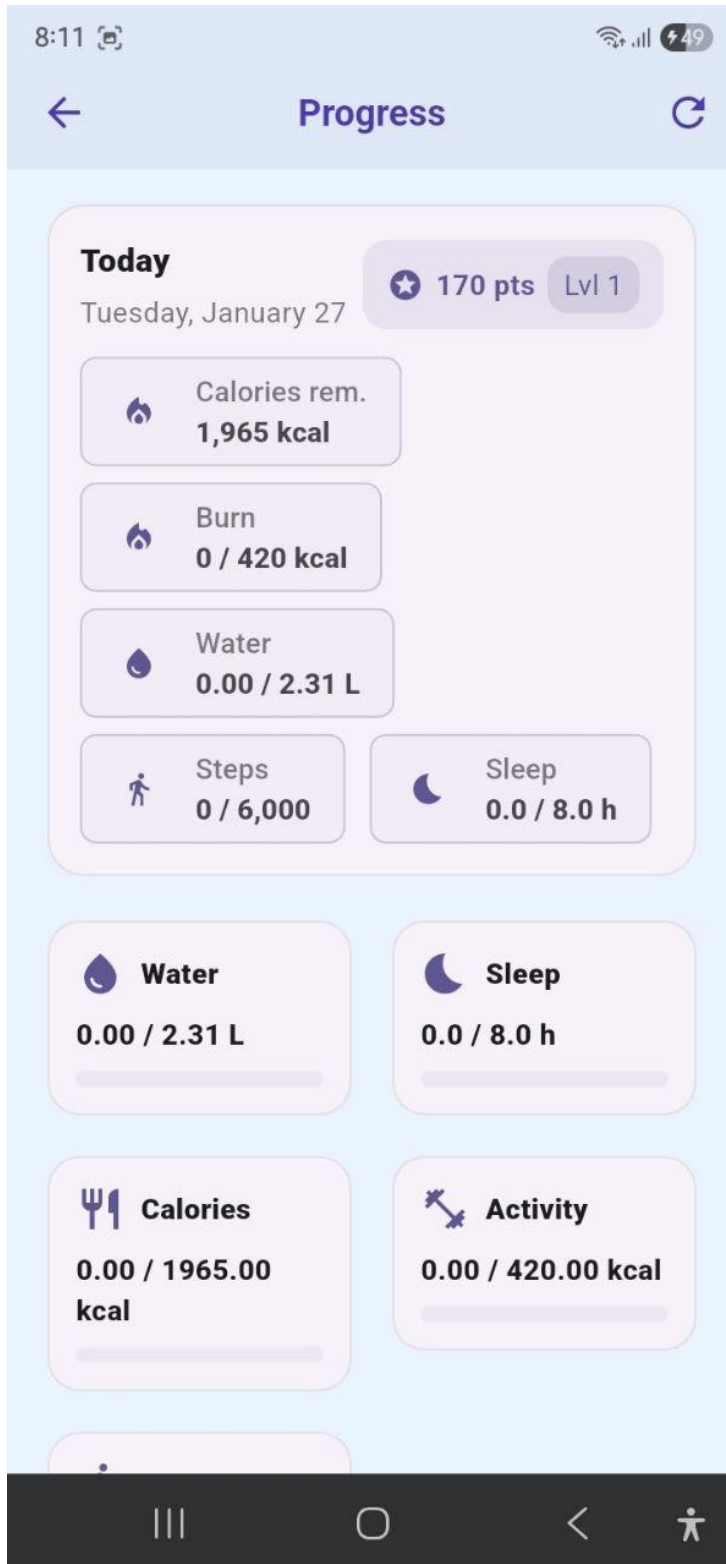
(Login screen)

- **EV-UI-02:**



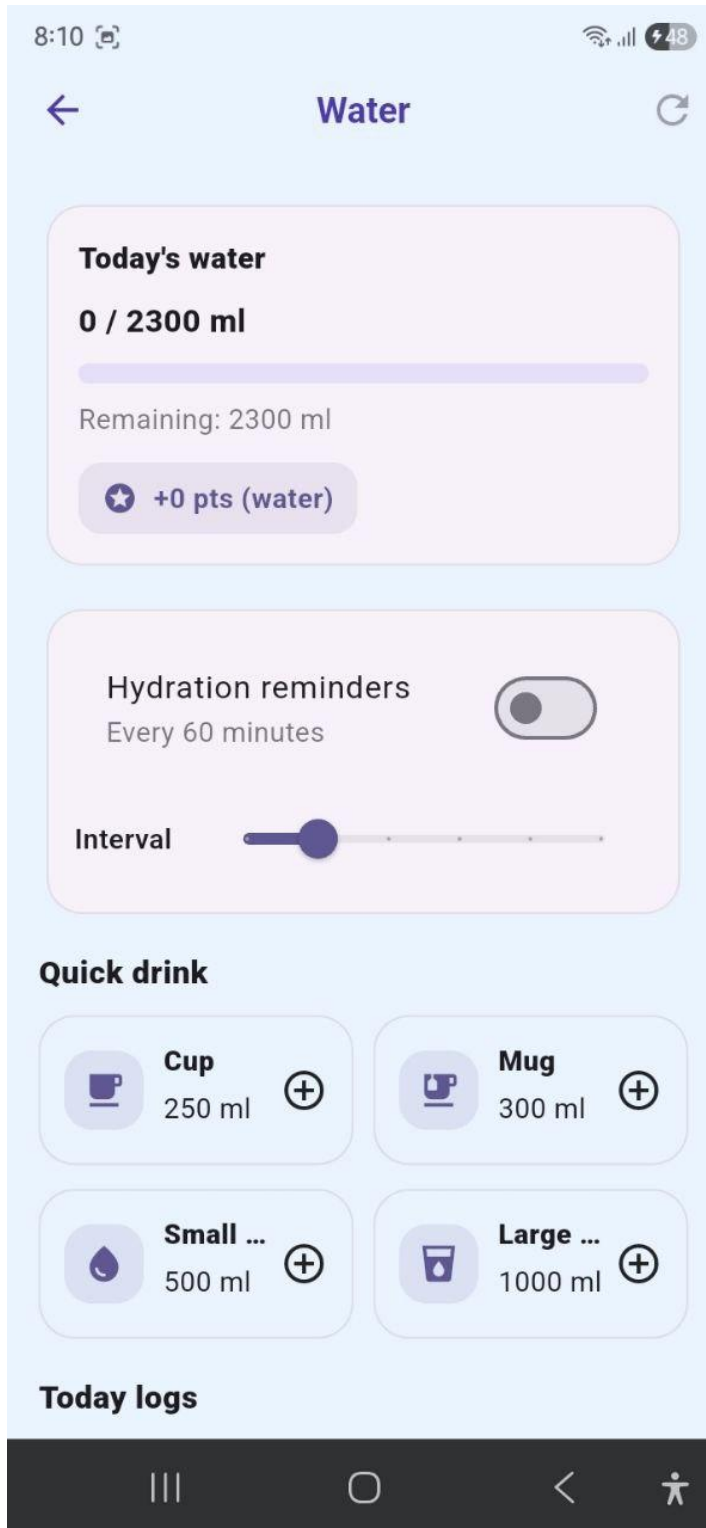
(Home/Dashboard overview)

- **EV-UI-03:**



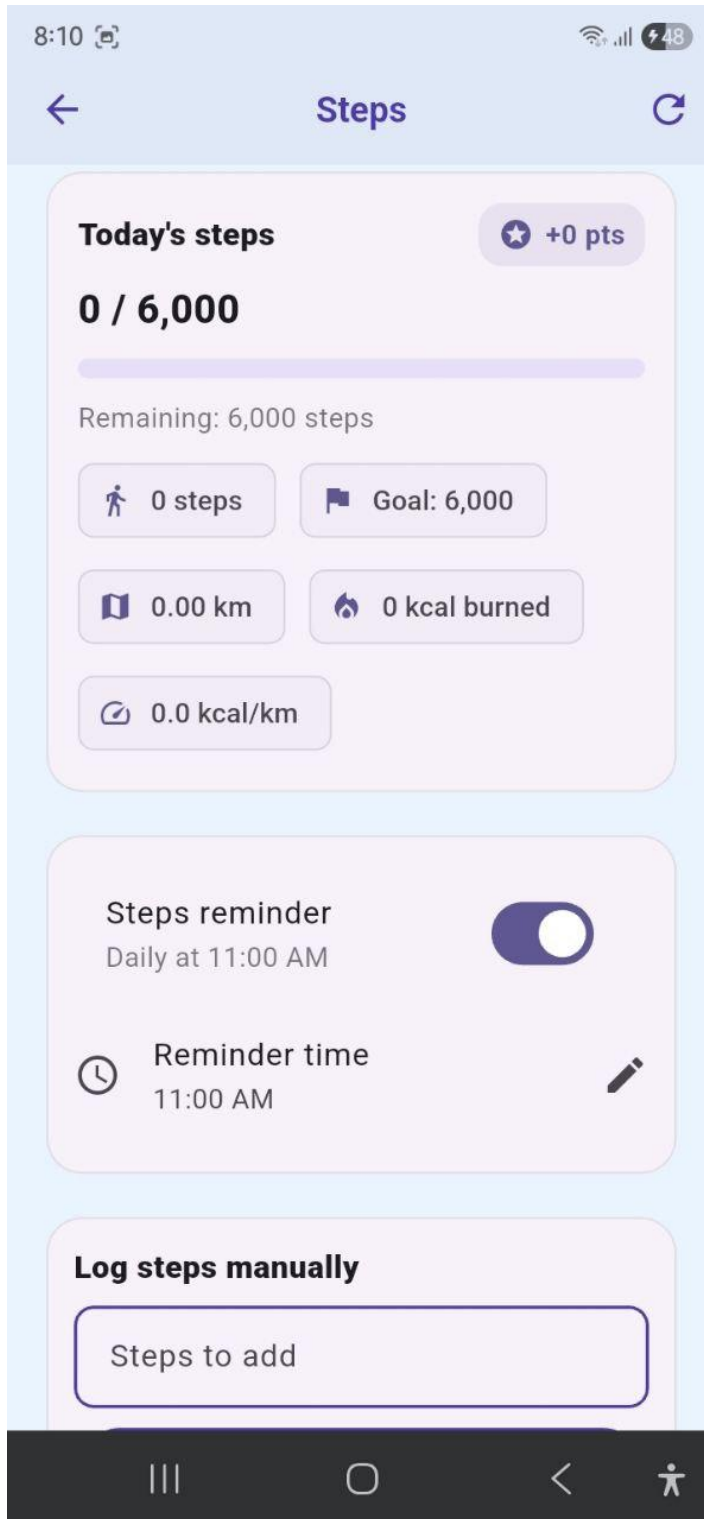
(Progress / summary view)

- **EV-UI-04:**



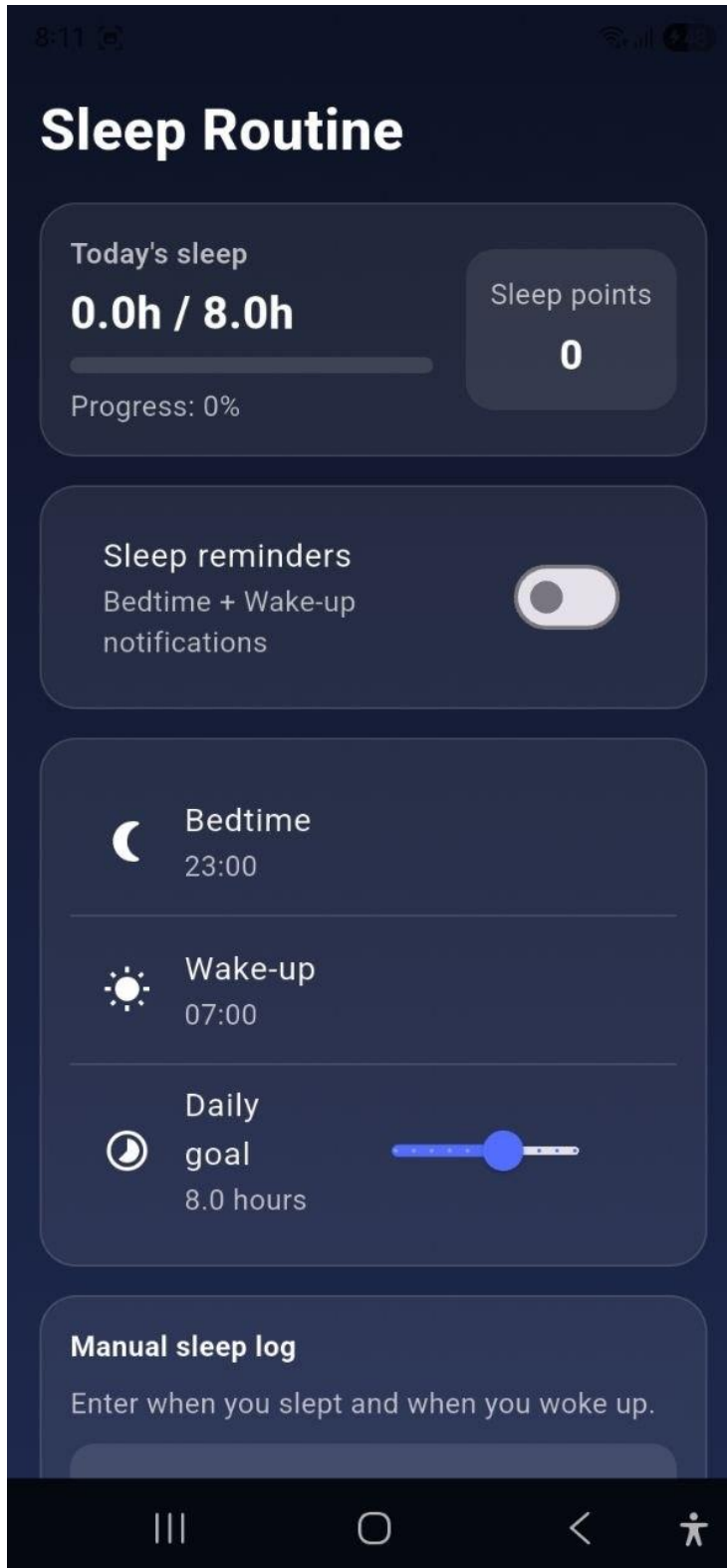
(Water tracking screen)

- **EV-UI-05:**



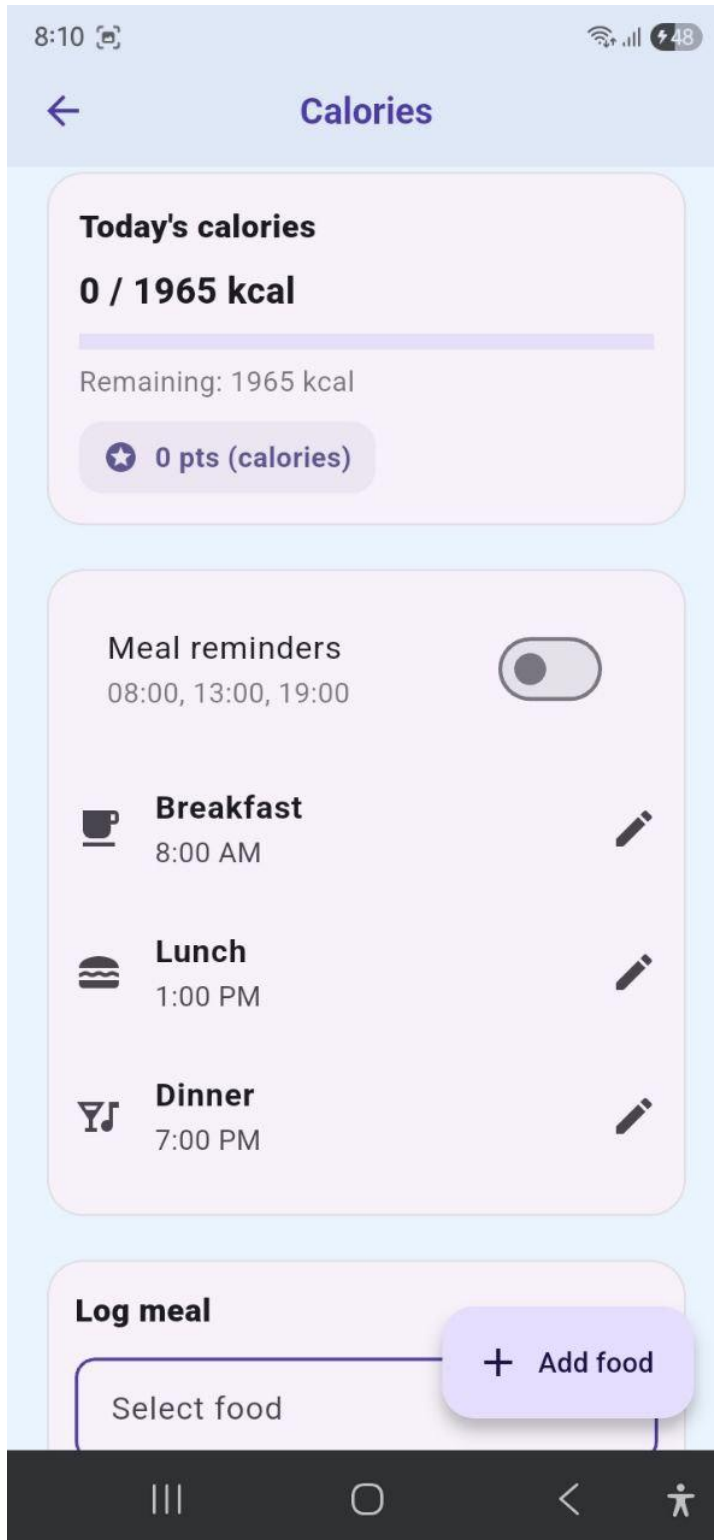
(Steps tracking screen + reminder UI)

- **EV-UI-06:**



(Sleep routine screen)

- **EV-UI-07:**



(Calories/Nutrition screen)

- **EV-UI-08:**

8:11 48

← **Activity**

Today's activity
1395 kcal burned
★ 25 pts (activity)

Activity reminders
Daily at 10:00 AM

Reminder time
10:00 AM

Log activity

Select exercise ▼

Duration (minutes) min

Save activity

(Activity screen)

6.6.9 Known Gaps, Limitations, and Future Improvements

- **Backend test discovery scope:** The captured backend evidence run reports “Found 5 test(s)”. While it confirms successful execution, it likely reflects partial discovery in the current environment. A future improvement is to standardize the backend test package layout (e.g., using users/tests/, core/tests/, gamification/tests/ and ensuring imports do not shadow tests) and re-capture evidence for the full suite.
- **Authentication endpoints performance:** Under local load conditions, register/login requests show higher latency compared to core daily endpoints. This is acceptable for infrequent operations and can be improved through production configuration, database tuning, and caching strategies.
- **Usability validation:** UI screenshots provide supporting evidence; a formal usability test (task completion time, number of steps, user feedback) can be added for stronger NFR-08/NFR-09 compliance.

6.6.10 Summary of Verification Outcomes

Automated verification demonstrates that the system operates correctly under core scenarios. Flutter tests confirm critical client-side behavior (authentication flow, secure storage interactions, permissions handling, and notification scheduling). Load testing confirms stable performance and concurrency for daily usage operations at 20 concurrent users with 0% failures and low p95 latencies for dashboard and logging endpoints. UI screenshots further confirm that implemented features are accessible and functional from the end-user perspective. Remaining limitations are documented and traceable for future refinement.

Conclusion & Future Work

VitaMate was designed and implemented as a mobile health companion that helps users manage daily wellness through integrated tracking of nutrition, hydration, sleep, and physical activity. The system combines a Flutter-based cross-platform mobile application with a Django/DRF backend, enabling secure user authentication, personalized profile-based targets, and consistent daily logging. The project also established a modular architecture on both backend and frontend to support maintainability and future evolution.

Throughout development, the scope was refined to prioritize core health-tracking workflows and ensure a stable end-to-end experience. As a result, the delivered system focuses on authentication, profile management, meals and food database, water tracking, sleep logging, steps tracking, dashboards, and Android notifications. Features such as GPS route tracking and administrative dashboards were intentionally excluded or deferred based on time and complexity constraints, and some areas remain partially implemented, including sleep improvement logic, full BMI classification output, and downloadable reporting.

Future work will focus on completing the partially implemented features, improving reporting capabilities (e.g., exportable weekly summaries), expanding cross-platform notification validation on iOS, strengthening production security configuration and deployment readiness, and enhancing scalability through pagination, indexing, and performance tuning based on measured load-testing results.